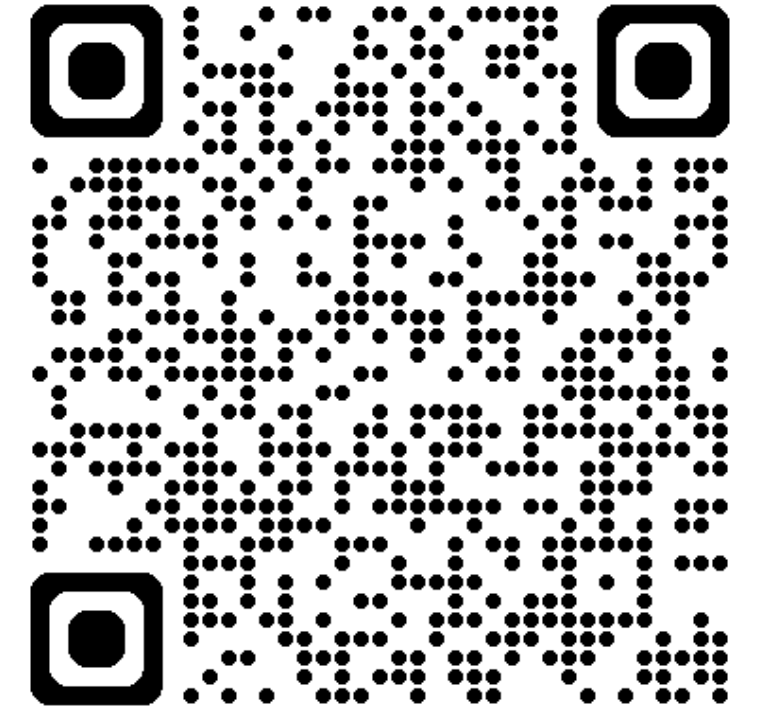


Übungsstunde 9



Heute:

- Repetitions Quiz
- kleiner Rekursions Recap
- **Dynamic Programming, wichtiges Thema!**

Repetitionsquiz

```
y = [(1,2),(3,4),(5,6)]
```

```
ls = [x**2 for z in y for x in z]
```

```
print(ls)
```



0.0s

Repetitionsquiz

```
y = [(1,2), (3,4), (5,6)]
```

```
ls = [x**2 for z in y for x in z]  
print(ls)
```

] ✓ 0.0s

```
[1, 4, 9, 16, 25, 36]
```

Repetitionsquiz

```
name = 'NAME NAME'  
x = name.split()  
print( x, type(x))
```



0.0s

Repetitionsquiz

```
name = 'NAME NAME'  
x = name.split()  
print( x, type(x))
```

✓ 0.0s

```
['NAME', 'NAME'] <class 'list'>
```

```
print(name.strip())
```

✓ 0.0s

Repetitionsquiz

```
] name = 'NAME NAME'  
x = name.split()  
print( x, type(x))
```

✓ 0.0s

['NAME', 'NAME'] <class 'list'>

```
✓ print(name.strip())
```

✓ 0.0s

NAME NAME

Frage: wie bekomme ich elegant den whitespace weg?

Repetitionsquiz

```
] name = 'NAME NAME'  
x = name.split()  
print( x, type(x))
```

✓ 0.0s

['NAME', 'NAME'] <class 'list'>

```
✓ print(name.strip())
```

✓ 0.0s

NAME NAME

Frage: wie bekomme ich elegant den whitespace weg?

Repetitionsquiz

```
print(name.replace(' ', ''))
```

1



0.0s

NAMENAME

Repetitionsquiz

```
import pandas as pd

# 1. Die CSV-Datei einlesen
df = pd.read_csv('produkte.csv')

# 2. Die ersten Zeilen der Daten anzeigen
print("--- Die ersten 5 Zeilen der Daten ---")
df.head()
```

✓ 0.0s

--- Die ersten 5 Zeilen der Daten ---

	Produkt	Preis	Bestand	Kategorie
0	Apfel	0.5	100	Obst
1	Orange	0.3	150	Obst
2	Kirsche	2.5	50	Obst
3	Dattel	1.2	20	Trockenfrucht
4	Erdbeere	3.0	80	Obst

```
# Was macht diese Zeile?? (1)
df['XXX'] = df['Preis'] * df['Bestand']

# Welche Werte haben value1/2?
value1 = df[df['Preis'] > 1.0]

value2 = df['Produkt'][3]
```

✓ 0.0s

Repetitionsquiz

```
print('wir haben in XX den totalen warenWert im lager des jeweiligen produktes berechnet')
df.head()
```

✓ 0.0s

wir haben in XX den totalen warenWert im lager des jeweiligen produktes berechnet

	Produkt	Preis	Bestand	Kategorie	XXX
0	Apfel	0.5	100	Obst	50.0
1	Orange	0.3	150	Obst	45.0
2	Kirsche	2.5	50	Obst	125.0
3	Dattel	1.2	20	Trockenfrucht	24.0
4	Erdbeere	3.0	80	Obst	240.0

```
print('value2:',value2)
print('Value1: nach preis über 1CHF gefiltert')
value1.head()
```

✓ 0.0s

value2: Dattel

Value1: nach preis über 1CHF gefiltert

	Produkt	Preis	Bestand	Kategorie	XXX
2	Kirsche	2.5	50	Obst	125.0
3	Dattel	1.2	20	Trockenfrucht	24.0
4	Erdbeere	3.0	80	Obst	240.0

Refresher Rekursion

Wurde öfters danach gefragt

Grundsätzlich ist Rekursion nicht ein Schwerpunkt in Inf2, aber aufgrund ihrer Verwendung für Sortieralgorithmen je nachdem vorausgesetzt und kann auch bei anderen Aufgaben vereinzelt vorkommen.

Rekursion kann schwierig sein, ich empfehle wirklich sich in der Lernphase gewisse rekursive Funktionen genau anzuschauen, um zu verstehen, was passiert.

MergeSort bsp wäre dafür perfekt geeignet, da habt ihr gleich doppelten Nutzen

Fakultätsfunktion

Verschiedene Wege

- Iterativ: „Ich multipliziere eine Sequenz von Zahlen 1-n miteinander auf“

```
def it_factulty(n):  
    factorial=1  
    for i in range(1,n+1): #+1 wegen exklusiven ende  
        factorial*=i  
    return factorial
```


Fakultätsfunktion

Verschiedene Wege

- Iterativ: „Ich multipliziere eine Sequenz von Zahlen 1-n miteinander auf“

```
def it_factulty(n):  
    factorial=1  
    for i in range(1,n+1): #+1 wegen exklusiven ende  
        factorial*=i  
    return factorial
```

Rekursiv: „Um das große Problem zu lösen, löse ich eine kleinere Version von mir selbst.“

Fakultätsfunktion

rekursiv

Rekursiv: „Um das große Problem zu lösen, löse ich eine kleinere Version von mir selbst.“

```
def rec_factorial(n):  
    if n <= 1:  
        return 1  
    else:  
        return n * rec_factorial(n-1)
```

Fakultät von 4, Rekursiv

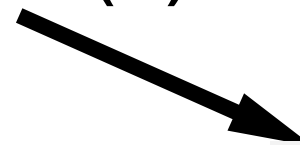
Der Prozess



originaler Funktionsaufruf `rec_factorial(4)`

```
rec_factorial(n=3):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

`3 * rec_factorial(2)` # `rec_factorial(2)` ist noch unbekannt!



```
rec_factorial(n=2):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

⬇ originaler Funktionsaufruf `rec_factorial(4)`

```
rec_factorial(n=3):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

$3 * \text{rec_factorial}(2)$

#`rec_factorial(2)` ist noch **unbekannt!**

```
rec_factorial(n=2):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

#`rec_factorial(1)` ist an dieser stelle
angenommen **unbekannt**

$2 * \text{rec_factorial}(1)$

```
rec_factorial(n=1):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

basisfall: wir kennen die Antwort!

⚡ originaler Funktionsaufruf `rec_factorial(4)`

```
rec_factorial(n=3):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

$3 * \text{rec_factorial}(2)$

#`rec_factorial(2)` ist noch **unbekannt!**

```
rec_factorial(n=2):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

#`rec_factorial(1)` ist an dieser stelle
angenommen **unbekannt**

$2 * \text{rec_factorial}(1)$

```
rec_factorial(n=1):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

basisfall: wir kennen die Antwort!

$2*1$

⬇ originaler Funktionsaufruf `rec_factorial(4)`

```
rec_factorial(n=3):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

$3 * \text{rec_factorial}(2)$

#`rec_factorial(2)` ist noch **unbekannt!**

```
rec_factorial(n=2):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

#`rec_factorial(1)` ist an dieser stelle
angenommen **unbekannt**

$2 * \text{rec_factorial}(1)$

```
rec_factorial(n=1):  
if n <= 1:  
    return 1  
else:  
    return n * rec_factorial(n-1)
```

basisfall: wir kennen die Antwort!

$3*2=6$

$2*1$

2

Das Prinzip ist immer dasselbe

- Den Computer „guiden“ das Problem stetig zu zerkleinern/auzuteilen sodass er irgendwann den Basisfall erreicht
- Wie gesagt, ich empfehle im Sommer bei Gelegenheit wirklich einmal Merge Sort, genau sich vorzustellen, was im Computer passiert. Us_7 Slides und .ipynb liefern bereits alles was ihr dazu braucht!

Fibonacci Zahl berechnen

```
def fibonacci(n):  
    if n == 1:  
        return 1  
    elif n == 2:  
        return 1  
    elif n > 2:  
        return fibonacci(n-1) + fibonacci(n-2)
```


Fibonacci Zahl berechnen

Dass Problem mit der Rekursion: Mehrfach das gleiche Problem(zahl) berechnen.

```
def fibonacci(n):  
    if n == 1:  
        return 1  
    elif n == 2:  
        return 1  
    elif n > 2:  
        return fibonacci(n-1) + fibonacci(n-2)
```

hier wird bsp die 2 Mehrfach „berechnet“

```
fibonacci(5) = fibonacci(4) + fibonacci(3)  
  
             = fibonacci(3) + fibonacci(2) + fibonacci(2) + fibonacci(1)  
  
             = fibonacci(2) + fibonacci(1) + fibonacci(2) + fibonacci(2) + fibonacci(1)  
  
             = 1 + 1 + 1 + 1 + 1
```

Mehrfach den selben Wert berechnen ist schlecht.

Die Lösung: Memoization

```
fibonacci_cache = {}

def fibonacci(n):
    # If we have cached the value, then return it
    if n in fibonacci_cache:
        return fibonacci_cache[n]

    # Compute the Nth term
    if n == 1:
        value = 1
    elif n == 2:
        value = 1
    elif n > 2:
        value = fibonacci(n-1) + fibonacci(n-2)

    # Cache the value and return it
    fibonacci_cache[n] = value
    return value
```

wir speichern jede neu berechnete Lösung im dictionary, sodass wir sie zukünftig gleich benutzen können

Fibonacci Dynamisch Iterativ Programmiert

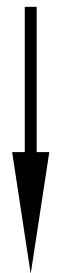
Das Problem mit Memoisierung: Wir sind immernoch rekursiv, heisst wir rufen eine Vielzahl von Funktionen gleichzeitig auf, dies kostet Speicher und ist somit Limitert & mit Kosten Verbunden

Iterativ haben wir diese Probleme nicht, wir sind viel effizienter unterwegs.

Fibonacci Dynamisch Iterativ Programmiert

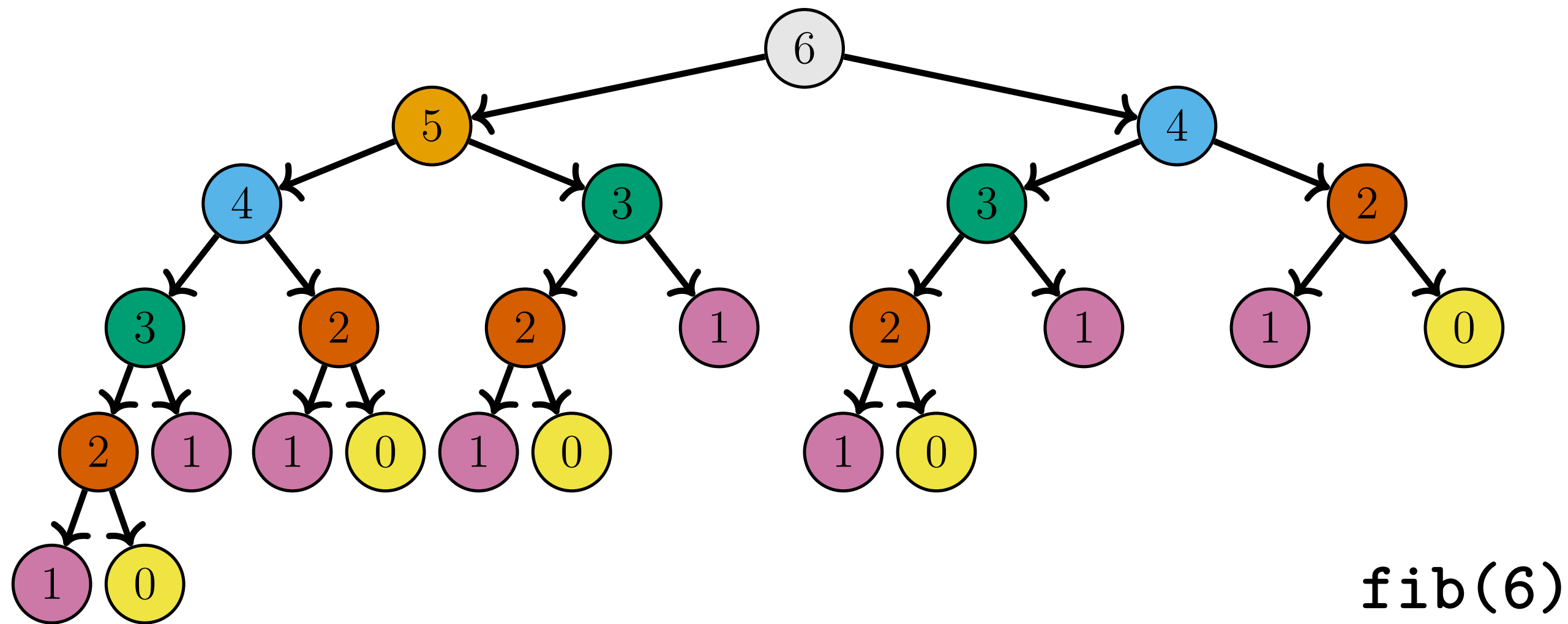
wir erstellen eine „Tabelle“ (1x n),
wo wir unsere fiboonaccis
speichern

```
def fibonacci_iterative_table(n):  
    if n <= 1:  
        return n  
  
    # Initialisierung der Tabelle (DP-Speicher)  
    # Wir erstellen eine Liste der Länge n + 1  
    dp = [0] * (n + 1)  
    dp[1] = 1  
  
    # Schrittweiser Aufbau der Lösung (Bottom-Up)  
    for i in range(2, n + 1):  
        dp[i] = dp[i-1] + dp[i-2]  
  
    return dp[n]
```

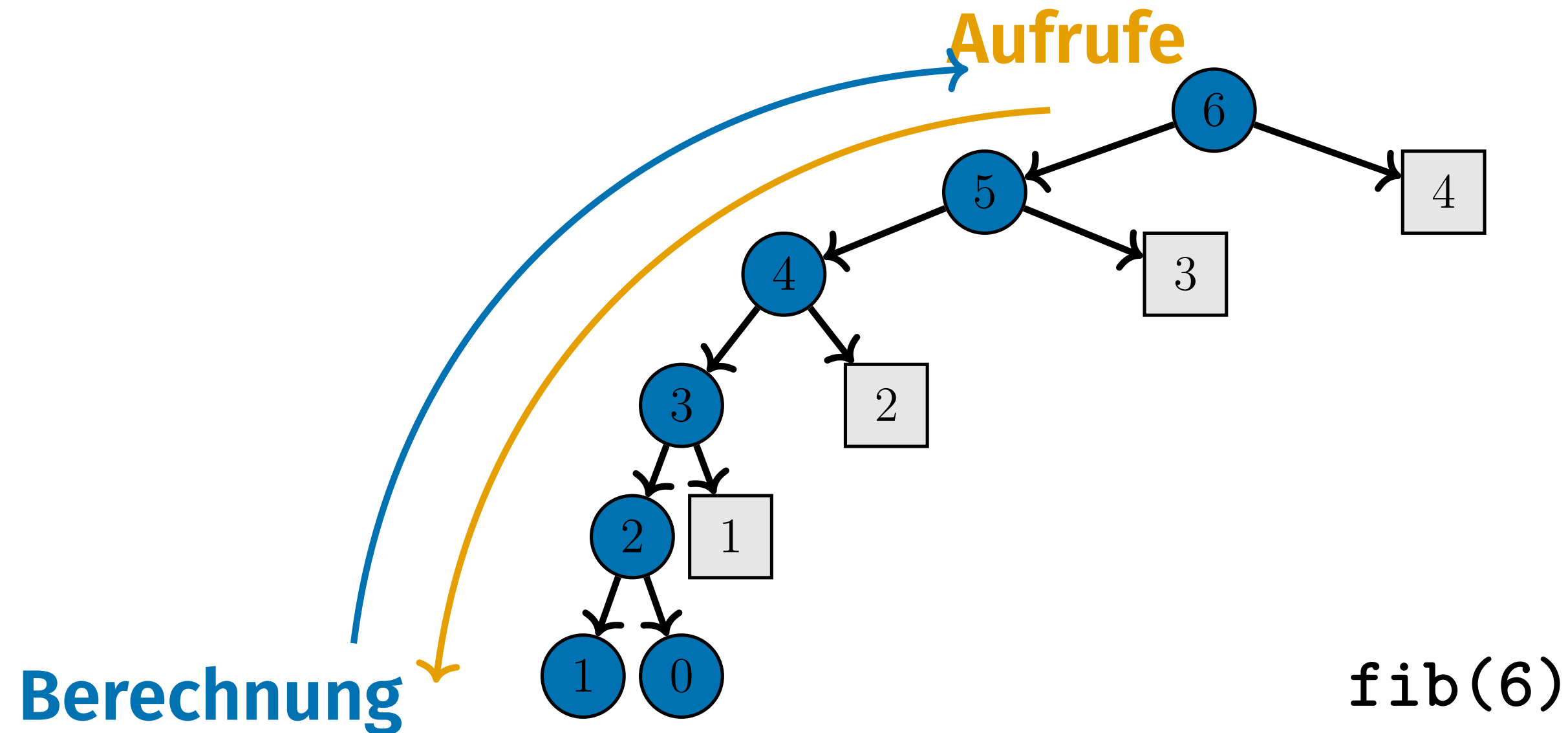
0	1	0	0	0	0
					
0	1	1	2	3	5

Wiederholung der Vorlesung

Probleme, die rekursiv gelöst werden können, wie Fibonacci, können wir mit Memoisierung (top-down) oder Dynamischer Programmierung (bottom-up) optimieren, indem wir redundante Berechnungen vermeiden.

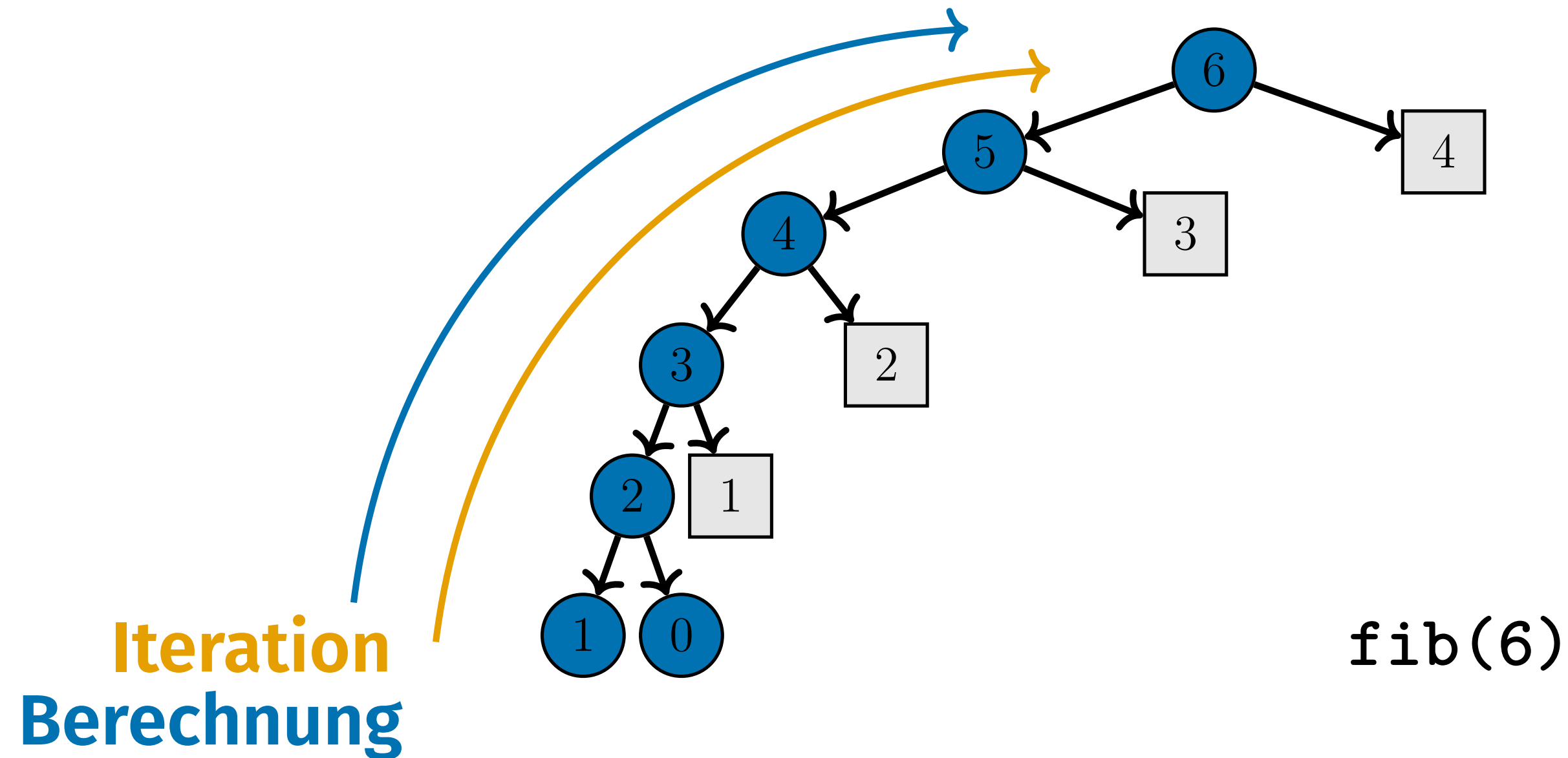


Memoisierung...



Memoisierung funktioniert, indem Lösungen zu Teilproblemen in einer Tabelle gespeichert werden. Diese Tabelle wird mit jedem **rekursiven Aufruf weitergegeben**, um redundante Berechnungen zu vermeiden, indem zuvor berechnete Ergebnisse (quadratische Knoten) wiederverwendet statt neu

...vs. Dynamische Programmierung: Tabellierung



Bottom-up dynamische Programmierung löst Probleme **iterativ** vom kleinsten Teilproblem aus aufwärts. Sie verwendet eine Tabelle, um die Ergebnisse zu speichern und redundante Berechnungen zu vermeiden.

Memoisierung != Dynamische Programmierung

Memoisierung

- **Ansatz?** Löst nur die benötigten Teilprobleme.
- **Rekursion?** Mit Rekursion implementiert, Tabelle wird mit jedem Aufruf weitergegeben.
- **Aufrufrichtung?** Top-down.

Dynamische Programmierung

Memoisierung != Dynamische Programmierung

Memoisierung

- **Ansatz?** Löst nur die benötigten Teilprobleme.
- **Rekursion?** Mit Rekursion implementiert, Tabelle wird mit jedem Aufruf weitergegeben.
- **Aufrufrichtung?** Top-down.

Dynamische Programmierung

- **Ansatz?** Löst alle Teilprobleme im Voraus, auch ungenutzte.
- **Rekursion?** Typischerweise iterativ implementiert.
- **"Aufruf"-Richtung?** Bottom-Up.

Memoisierung != Dynamische Programmierung

Memoisierung

- **Ansatz?** Löst nur die benötigten Teilprobleme.
- **Rekursion?** Mit Rekursion implementiert, Tabelle wird mit jedem Aufruf weitergegeben.
- **Aufrufrichtung?** Top-down.

Dynamische Programmierung

- **Ansatz?** Löst alle Teilprobleme im Voraus, auch ungenutzte.
- **Rekursion?** Typischerweise iterativ implementiert.
- **"Aufruf"-Richtung?** Bottom-Up.

Beide Taktiken lösen dasselbe Problem, aber auf unterschiedliche Weise. Eine Bottom-Up-Lösung ist eleganter und füllt den Call Stack nicht, während Memoisierung schnell implementiert ist, ohne das Problem im Detail zu betrachten.

Bemerkung: Namensgebung

- Grundsätzlich wird mit dynamischer Programmierung **oft auch Memoization gemeint.**
- **In INF2: wird mit „dynamischer Programmierung“ die iterative Lösung gemeint.** (und Memoization ist einfach Memoization)

2. Königsweg

Aufgabenbeschreibung: Königsweg

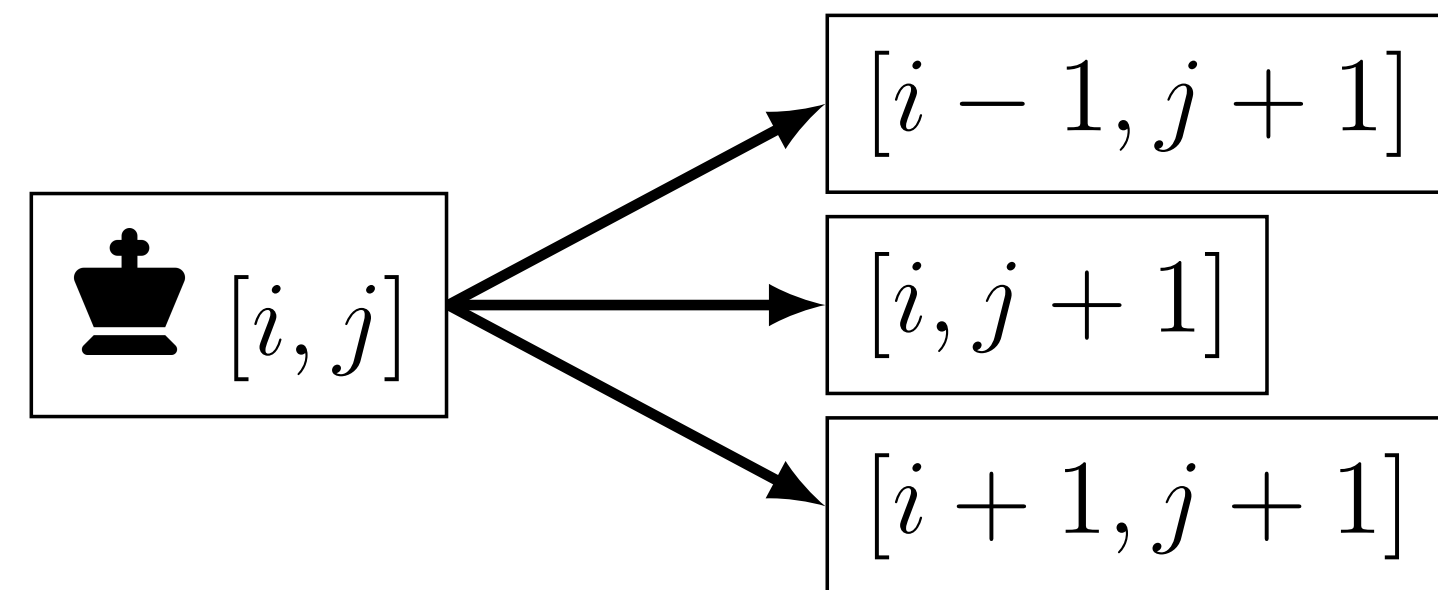
- Gegeben: Ein Schachbrett a mit dem König startend auf Zelle $[0, 0]$, wobei jede Zelle eine nicht-negative Belohnung (0 oder mehr) enthält.

2	1	3	0	1
0	1	0	1	1
0	1	0	0	0

Aufgabenbeschreibung: Königsweg

- Gegeben: Ein Schachbrett a mit dem König startend auf Zelle $[0, 0]$, wobei jede Zelle eine nicht-negative Belohnung (0 oder mehr) enthält.
- Auf jeder Zelle hat der König drei Optionen, um sich von Position $[i, j]$ zu bewegen, solange er das Schachbrett nicht verlässt.

	1	3	0	1
0	1	0	1	1
0	1	0	0	0

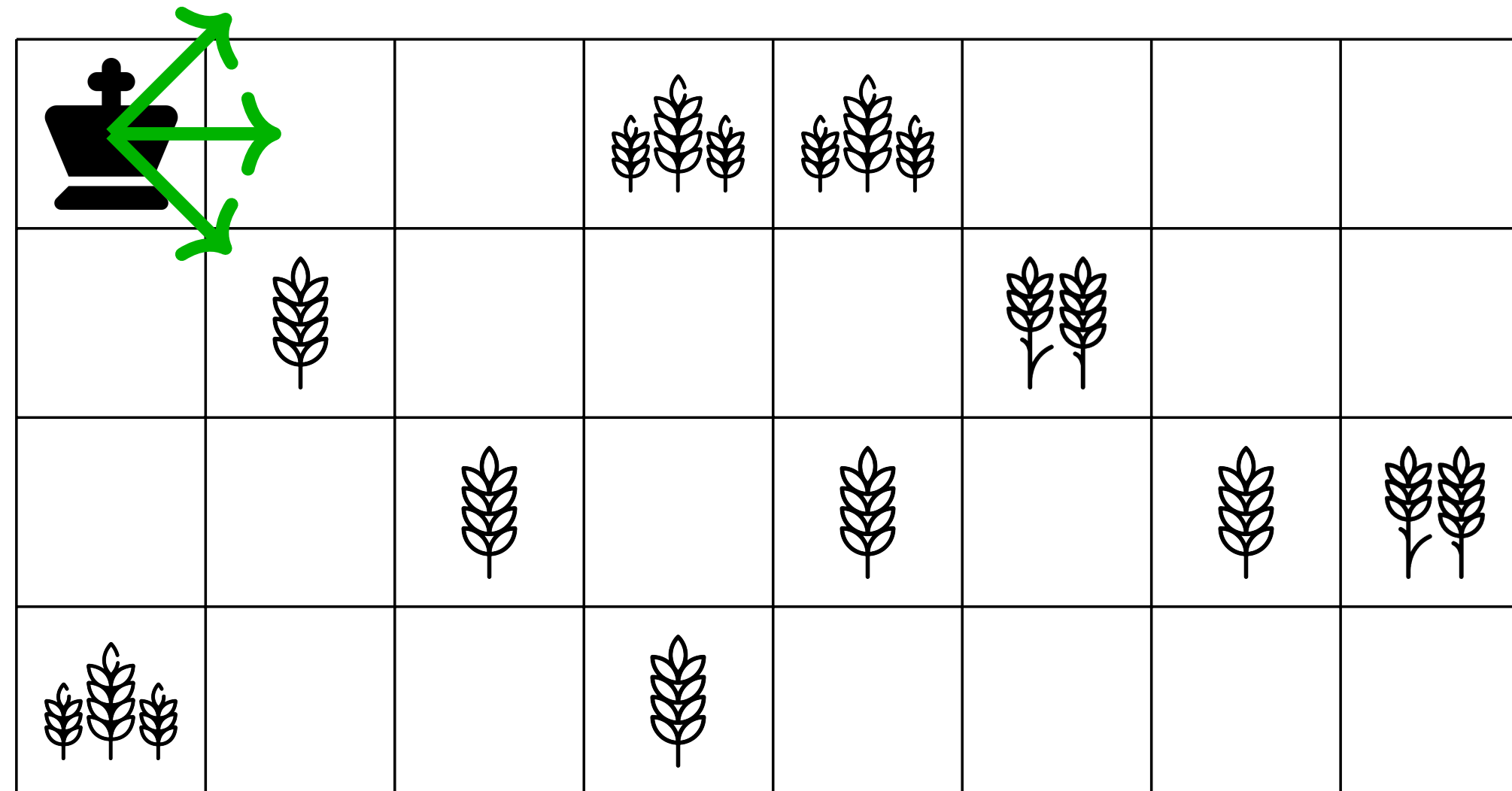


Aufgabenbeschreibung: Königsweg

- **Eingabe:** Ein Schachbrett a als 2D-Array der Grösse $n \times m$, wobei jede Zelle eine nicht-negative ganze Zahl enthält, welche die entsprechende Belohnung angibt.
- **Ziel:** Der König startet auf Zelle $[0, 0]$ und kann sich in drei Richtungen bewegen: nord-östlich, östlich und süd-östlich. Das Ziel ist es, den optimalen Pfad zu finden, um die Gesamtbelohnung zu maximieren, während der König sich zur rechten Seite bewegt.
- **Ausgabe:** Die maximale Belohnung, die der König auf seinem Weg zur rechten Seite des Schachbretts einsammeln kann.

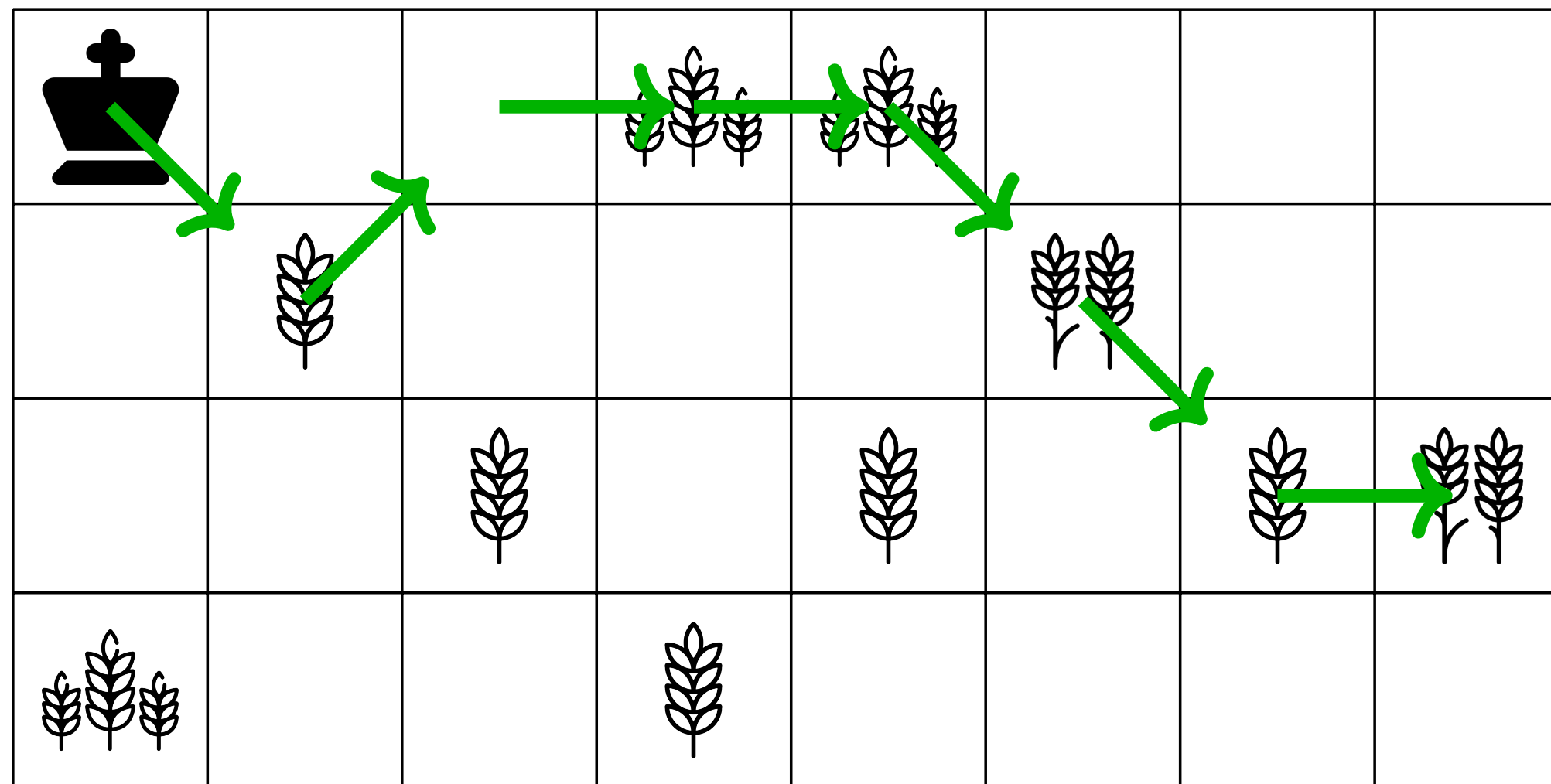
Aufgabenbeschreibung: Königsweg Beispiel

Was ist die maximale Belohnung, die der König einsammeln kann?

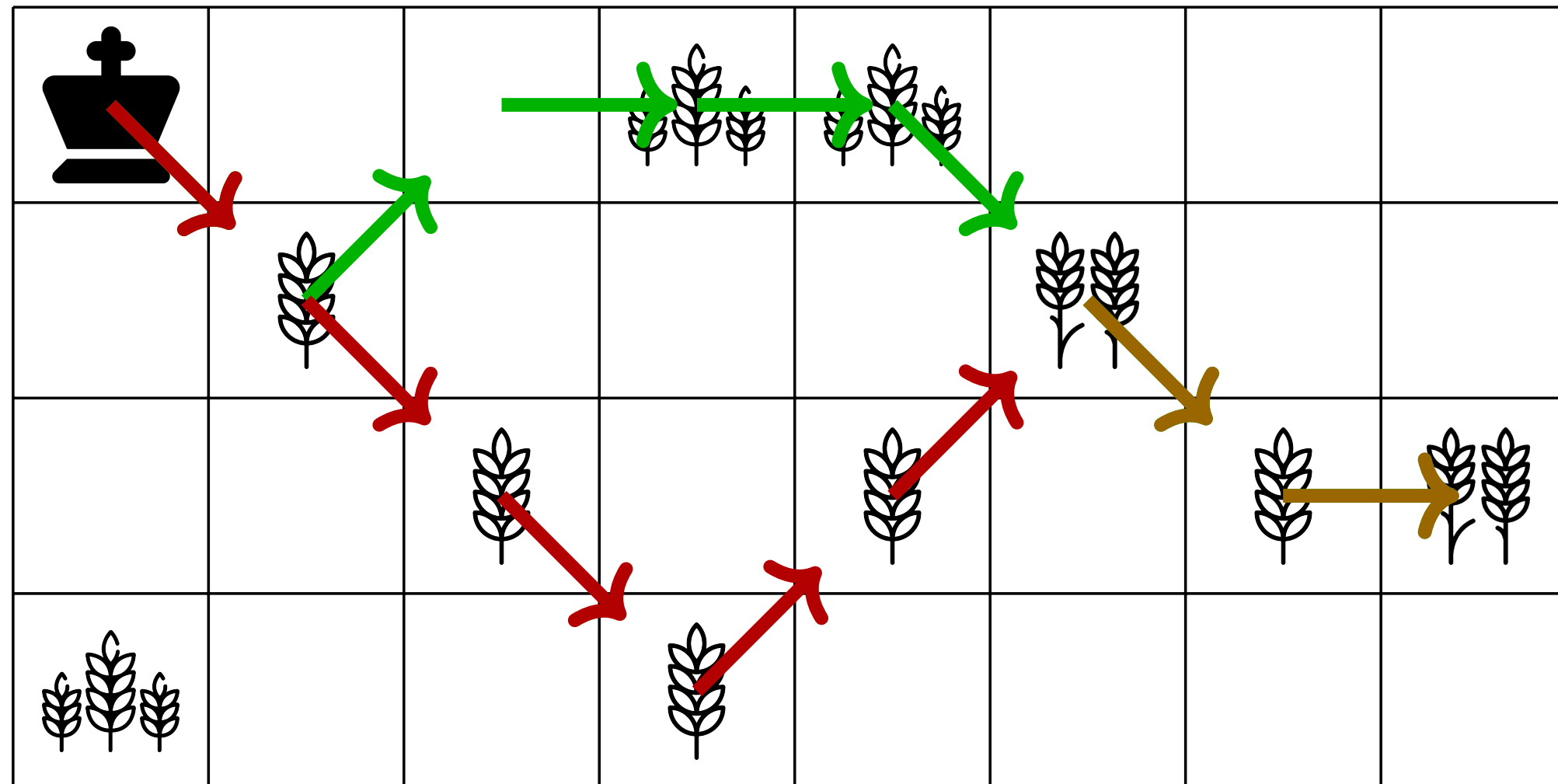


Aufgabenbeschreibung: Königsweg Beispiel

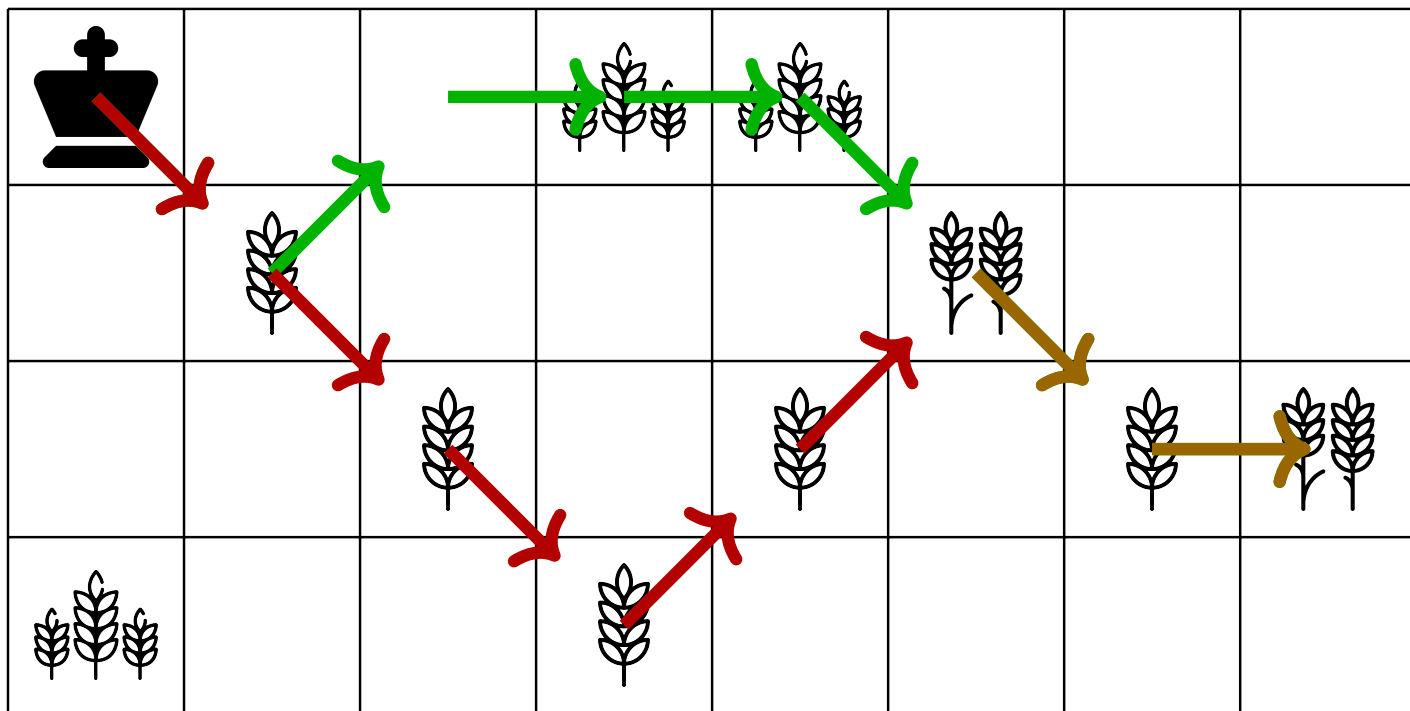
Was ist die maximale Belohnung, die der König einsammeln kann? **12 Weizen**



Greedy Lösung

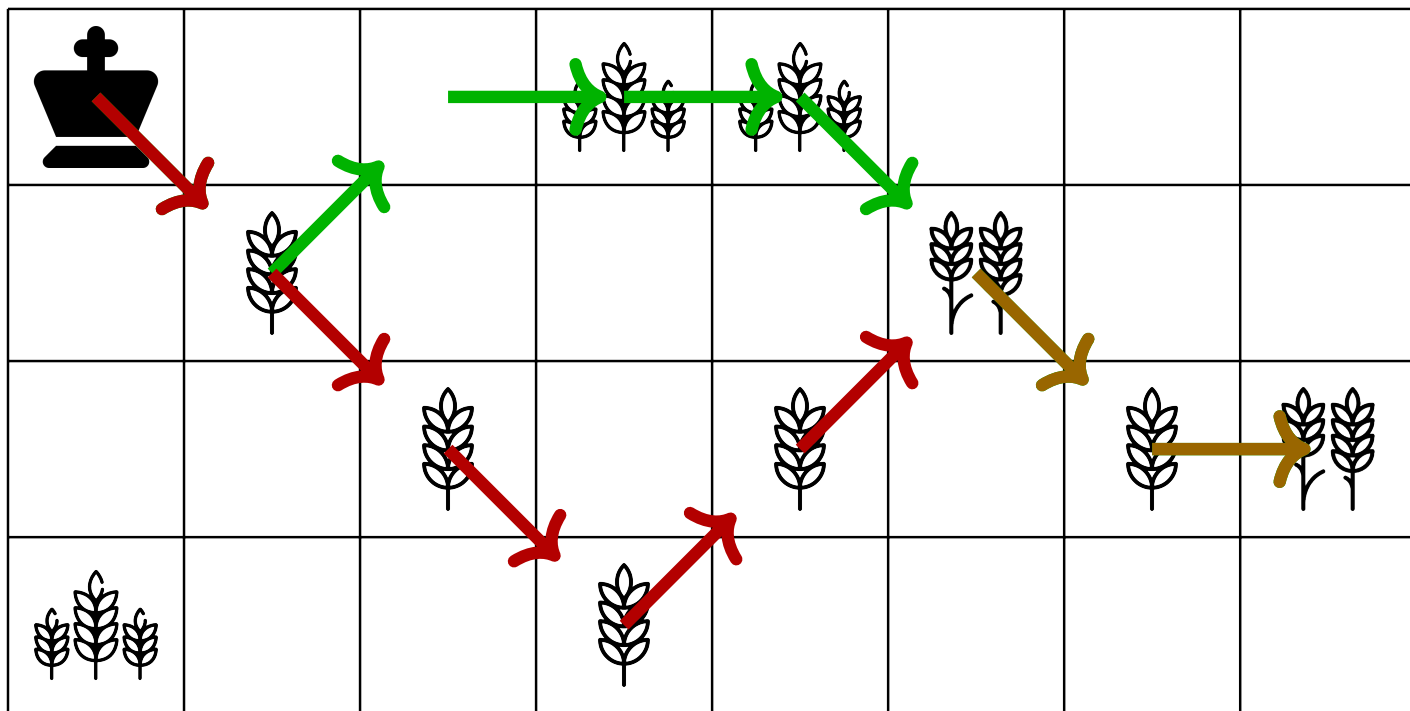


Greedy Lösung



- Nur die sofortige Belohnung zu nehmen, kann zu suboptimalen Entscheidungen führen, welche grössere Belohnungen weiter vorne verpassen.

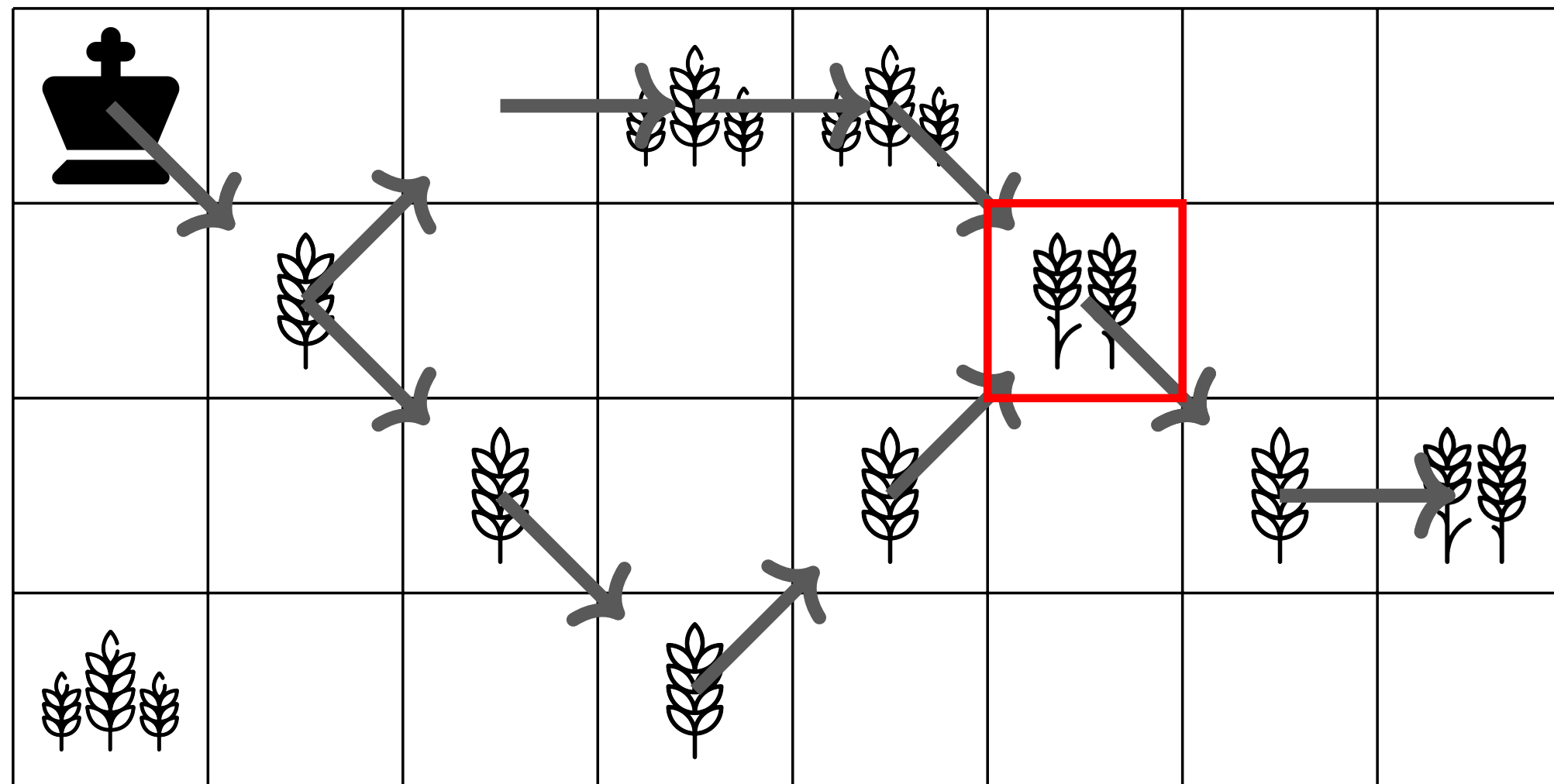
Greedy Lösung



- Nur die sofortige Belohnung zu nehmen, kann zu suboptimalen Entscheidungen führen, welche grössere Belohnungen weiter vorne verpassen.
- Daher müssen wir jeden möglichen Pfad evaluieren, um den global optimalen Pfad zu finden.

Überlappende Teilprobleme

Da mehrere Pfade zur gleichen Zelle führen können, wird das Teilproblem mehrfach gelöst, was zu redundanten Berechnungen führt. Im Beispiel gibt es zwei Pfade, die an der markierten Zelle zusammenlaufen.



Warum ist DP nützlich?

- **Brute Force:** Das Vergleichen aller Pfade hat eine exponentielle Zeitkomplexität von $\mathcal{O}(3^{n \times m})$, was ineffizient ist.
- **Dynamische Programmierung** zerlegt das Problem in kleinere Teilprobleme und speichert Lösungen zur Wiederverwendung.
- **Zeitkomplexität:** $\mathcal{O}(n \times m)$, da wir eine Berechnung von $\mathcal{O}(1)$ für jede Zelle im $n \times m$ Gitter durchführen.
- **Effizienz:** DP vermeidet redundante Berechnungen, wodurch es signifikant effizienter wird.

Die drei Schritte der DP

1. **Identifiziere die Teilprobleme** und analysiere, wie sie sich überlappen.
Hier: Mehrere Pfade, die auf derselben Zelle zusammenlaufen.

Die drei Schritte der DP

1. **Identifiziere die Teilprobleme** und analysiere, wie sie sich überlappen.
Hier: Mehrere Pfade, die auf derselben Zelle zusammenlaufen.
2. **Definiere die rekursive Formel** durch Betrachtung möglicher Übergänge zwischen Teilproblemen und optimaler Entscheidungen.
Hier: Bewegungen des Königs.

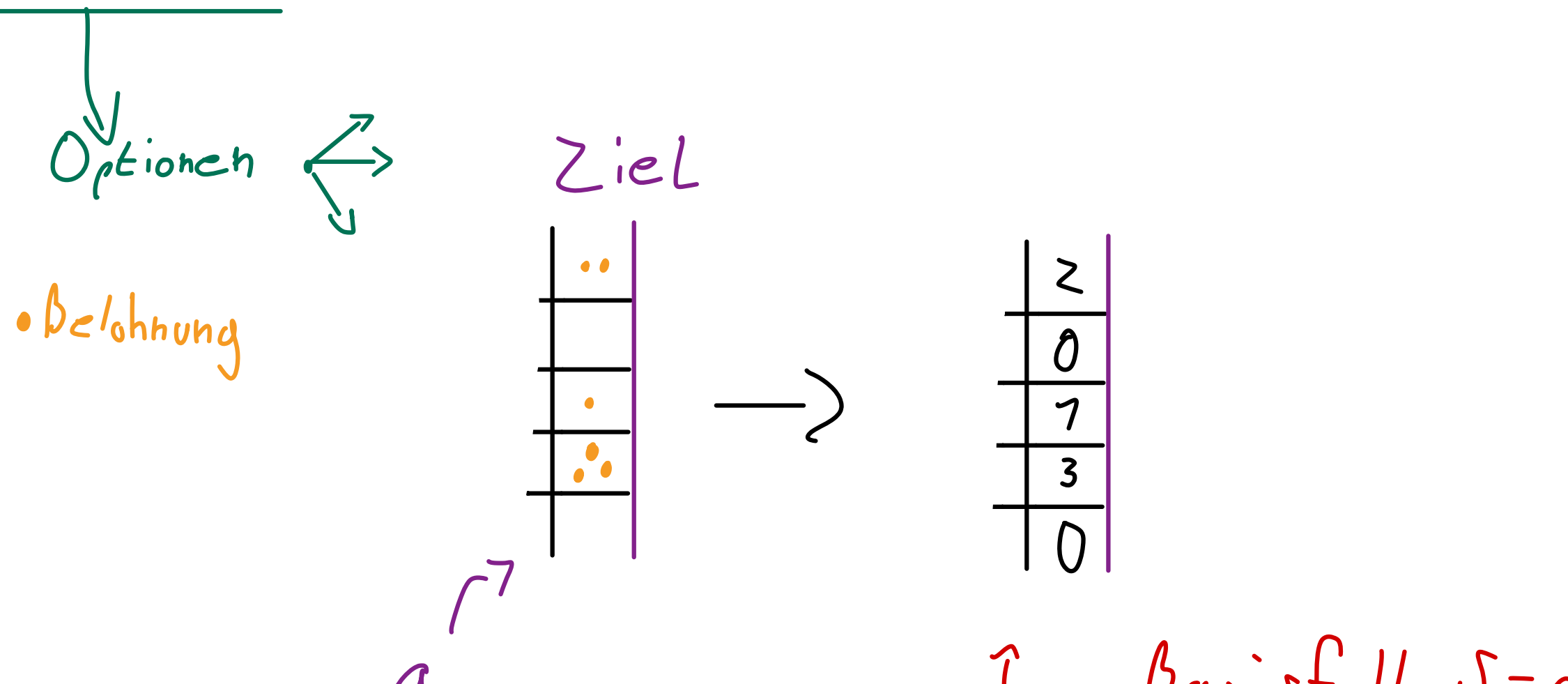
Die drei Schritte der DP

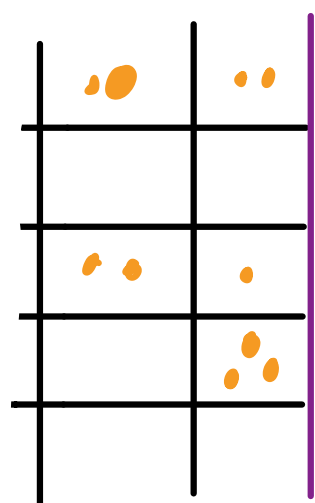
1. **Identifiziere die Teilprobleme** und analysiere, wie sie sich überlappen.
Hier: Mehrere Pfade, die auf derselben Zelle zusammenlaufen.
2. **Definiere die rekursive Formel** durch Betrachtung möglicher Übergänge zwischen Teilproblemen und optimaler Entscheidungen.
Hier: Bewegungen des Königs.
3. **Implementiere eine Lösung** mit einer geeigneten Datenstruktur und der korrekten Berechnungsreihenfolge. *Die Struktur ist fast immer eine Art Array oder Matrix.*

Schritt 1: Identifiziere das Teilproblem

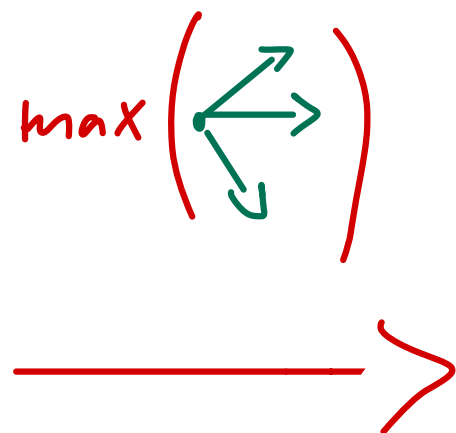
■ Was ist das Teilproblem?

Für jede Zelle $[i, j]$ auf dem Schachbrett ($i < n, j < m$) repräsentiert $S[i, j]$ die maximale Belohnung, die der König einsammeln kann, indem er sich von dieser Zelle zur letzten Spalte $j = m - 1$ bewegt und dabei alle möglichen Züge berücksichtigt.





a



$\mathcal{J} =$

4	2
2	0
5	1
3	3
3	0

Bottom up
←

+2	2
+0	0
+2	1
+0	3
+0	0

B, C, D

Schritt 1: Identifiziere das Teilproblem

- Was ist das Teilproblem?

Für jede Zelle $[i, j]$ auf dem Schachbrett ($i < n$, $j < m$) repräsentiert $S[i, j]$ die maximale Belohnung, die der König einsammeln kann, indem er sich von dieser Zelle zur letzten Spalte $j = m - 1$ bewegt und dabei alle möglichen Züge berücksichtigt.

- Welche Optionen hat der König an Position $[i, j]$?

- Bewege nach Nord-Ost $[i - 1, j + 1]$, falls $i > 0$
- Bewege nach Ost $[i, j + 1]$
- Bewege nach Süd-Ost $[i + 1, j + 1]$, falls $i < n - 1$

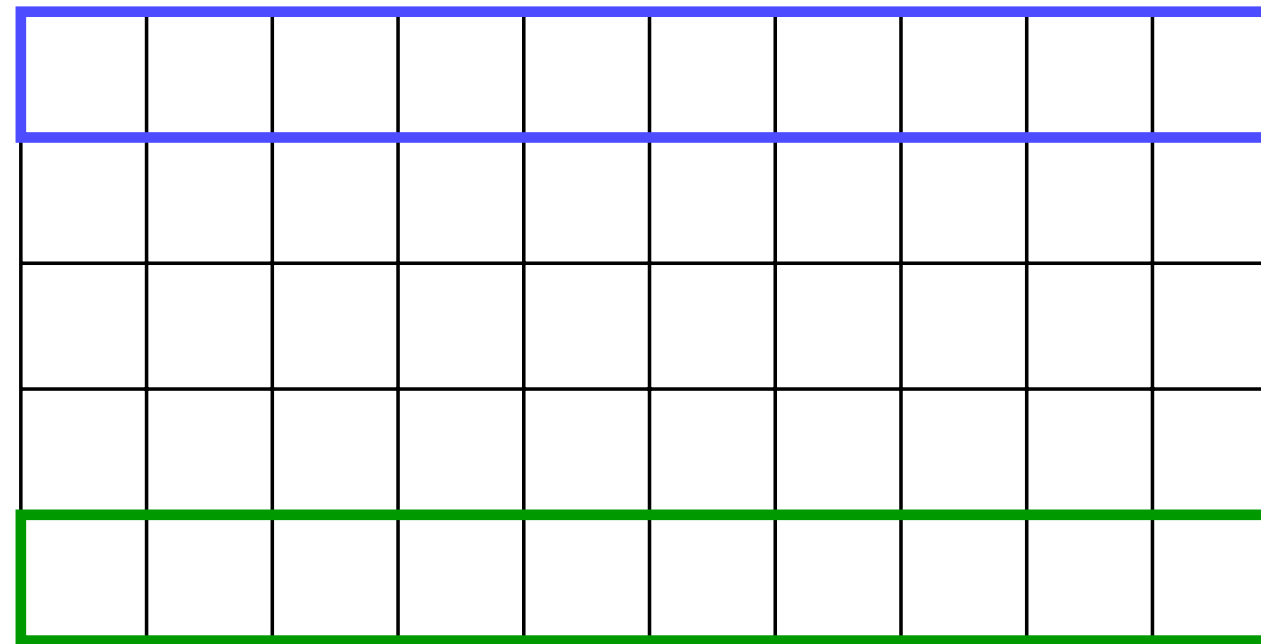
Randbedingungen

Kann der König sich immer nach Nord-Ost oder Süd-Ost bewegen?

Nein, Randbedingungen müssen berücksichtigt werden!

- Oberste Zeile ($i = 0$): Der König kann sich nicht nach Nord-Ost bewegen.
- Unterste Zeile ($i = n - 1$): Der König kann sich nicht nach Süd-Ost bewegen.

wichtig beim coden!



Basisfall

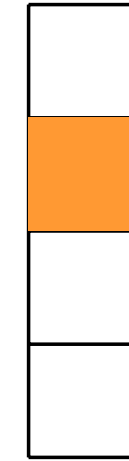
Sind das die einzigen Einschränkungen, denen der König auf dem Brett begegnet?

Nein, eine weitere zentrale Einschränkung ist, dass der König anhalten muss, wenn er die äusserste rechte Spalte ($j = m - 1$) erreicht, da darüber hinaus keine weiteren Züge möglich sind. Dies ist der **Basisfall**: Der König kann sich nicht weiterbewegen, also gilt $S[i, j] = a[i, j]$.



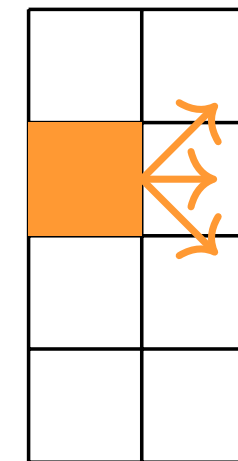
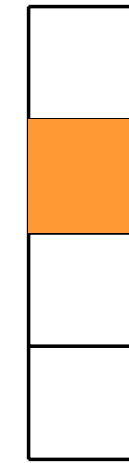
Schritt 1: Identifiziere die Teilprobleme

- Eine Spalte → Der König erhält die Belohnung der Zelle (Basisfall).



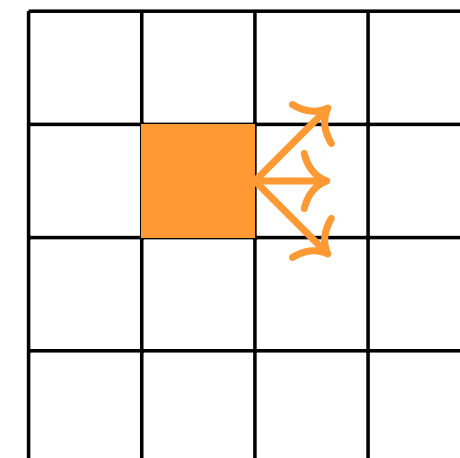
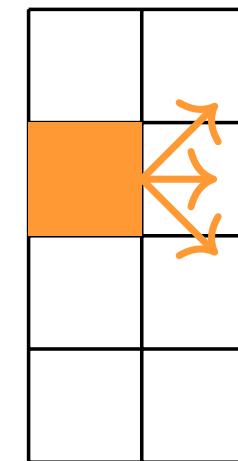
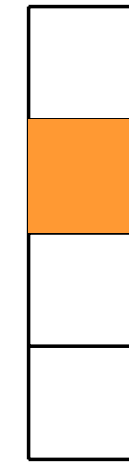
Schritt 1: Identifiziere die Teilprobleme

- Eine Spalte → Der König erhält die Belohnung der Zelle (Basisfall).
- Zwei Spalten → Der König hat drei mögliche Züge: nord-östlich, östlich und süd-östlich. Er wählt den Zug mit der maximalen Belohnung.



Schritt 1: Identifiziere die Teilprobleme

- Eine Spalte → Der König erhält die Belohnung der Zelle (Basisfall).
- Zwei Spalten → Der König hat drei mögliche Züge: nord-östlich, östlich und süd-östlich. Er wählt den Zug mit der maximalen Belohnung.
- Ganzes Schachbrett → Der König hat die gleichen drei Optionen. Aber um zu wissen, welche die beste ist, muss er zuerst die maximale Belohnung für alle drei Teilprobleme evaluieren.



Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Eine Matrix s der Grösse $n \times m$, wobei $S[i, j]$ die maximale Belohnung speichert, die eingesammelt werden kann, wenn man bei $[i, j]$ startet, für alle $i < n$ und $j < m$.

Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Eine Matrix s der Grösse $n \times m$, wobei $S[i, j]$ die maximale Belohnung speichert, die eingesammelt werden kann, wenn man bei $[i, j]$ startet, für alle $i < n$ und $j < m$.

- Um $S[i, j]$ zu berechnen, welche Teilprobleme müssen zuerst gelöst werden?

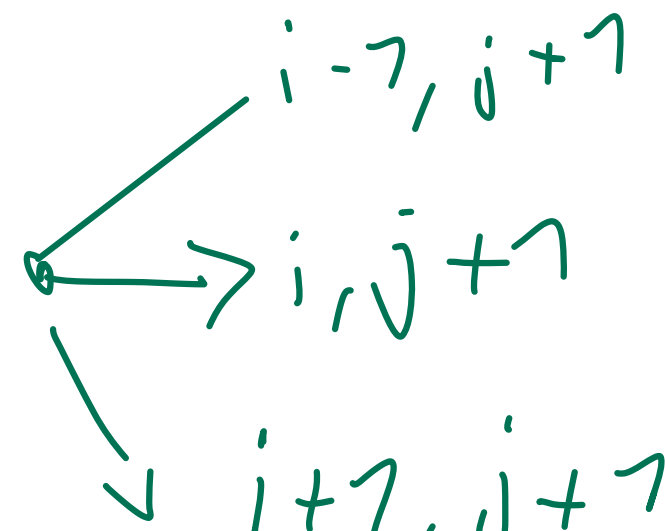
Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Eine Matrix s der Grösse $n \times m$, wobei $S[i, j]$ die maximale Belohnung speichert, die eingesammelt werden kann, wenn man bei $[i, j]$ startet, für alle $i < n$ und $j < m$.

- Um $S[i, j]$ zu berechnen, welche Teilprobleme müssen zuerst gelöst werden?

$S[i, j + 1]$, $S[\underline{i - 1}, j + 1]$ wenn $i > 0$, und $S[\underline{i + 1}, j + 1]$ wenn $j < m - 1$, wobei die Gittergrenzen respektiert werden.



Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Eine Matrix s der Grösse $n \times m$, wobei $S[i, j]$ die maximale Belohnung speichert, die eingesammelt werden kann, wenn man bei $[i, j]$ startet, für alle $i < n$ und $j < m$.

- Um $S[i, j]$ zu berechnen, welche Teilprobleme müssen zuerst gelöst werden?

$S[i, j + 1]$, $S[i - 1, j + 1]$ wenn $i > 0$, und $S[i + 1, j + 1]$ wenn $j < m - 1$, wobei die Gittergrenzen respektiert werden.

- In welcher Reihenfolge soll S anschliessend ausgefüllt werden?

Schritt 3: Implementierung

- Welche Datenstruktur ist am besten geeignet, um die Teilprobleme zu speichern?

Eine Matrix s der Grösse $n \times m$, wobei $S[i, j]$ die maximale Belohnung speichert, die eingesammelt werden kann, wenn man bei $[i, j]$ startet, für alle $i < n$ und $j < m$.

- Um $S[i, j]$ zu berechnen, welche Teilprobleme müssen zuerst gelöst werden?

$S[i, j + 1]$, $S[i - 1, j + 1]$ wenn $i > 0$, und $S[i + 1, j + 1]$ wenn $j < m - 1$, wobei die Gittergrenzen respektiert werden.

- In welcher Reihenfolge soll S anschliessend ausgefüllt werden?

Von rechts nach links, Spalte für Spalte. Die Reihenfolge, in der eine Spalte ausgefüllt wird, spielt keine Rolle.

DP-Tabelle

Während des DP-Übergangs müssen Randbedingungen beachtet werden:

- **DP-Tabellengrösse:** Wir verwenden eine DP-Tabelle mit derselben Grösse wie das Schachbrett, um die optimale Lösung für jede Zelle zu speichern.
- **Durchlauf:** Wir müssen die DP-Tabelle so durchlaufen, dass sichergestellt ist, dass alle Teilprobleme gelöst sind, bevor wir das aktuelle Problem berechnen.
- **Randbedingungen:**
 - Bei $j = m - 1$ (letzte Spalte) kann der König sich nicht weiter nach rechts bewegen. Dies ist der Basisfall, bei dem keine weitere Bewegung möglich ist und die Lösung unkompliziert ist.
 - Wenn $i = 0$ (oberste Zeile), ist der Zug nord-ost nicht möglich.
 - Wenn $i = n - 1$ (unterste Zeile), ist der Zug süd-ost nicht möglich.
 - An Randpositionen sollten nur die zulässigen Züge berücksichtigt werden.

Datenstruktur

Durch die rekursive Formulierung haben wir die Abhängigkeiten für die Berechnung der Lösung des Königswegs an Position $[i, j]$ definiert.

$$S[i, j] = \begin{cases} a[i, j] & j = m - 1 \\ a[i, j] + \max(S[i - 1, j + 1], S[i, j + 1]) & i = n - 1, j < m - 1 \\ a[i, j] + \max(S[i, j + 1], S[i + 1, j + 1]) & i = 0, j < m - 1 \\ a[i, j] + \max \begin{cases} S[i - 1, j + 1] \\ S[i, j + 1] \\ S[i + 1, j + 1] \end{cases} & 0 < i < n - 1, j < m - 1 \end{cases}$$

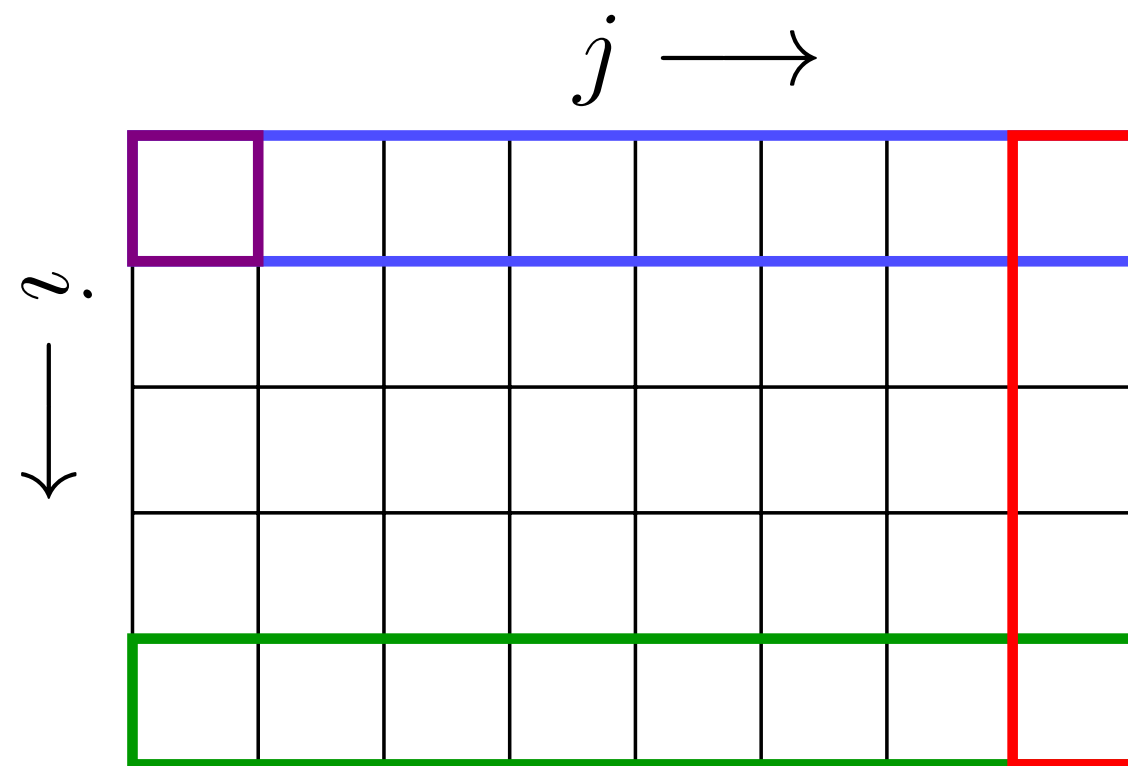
nur Notation: das grafische Konzept ist viel wichtiger!

Die rekursive Lösung in der Tabelle

- Sei $S[i, j]$ die maximale Belohnung, wenn der König beginnt, sich von dieser Zelle aus nach rechts zu bewegen.
 - Nachdem die ganze Tabelle berechnet wurde, könnten wir den König auf ein beliebiges Feld setzen und wüssten, welche maximale Belohnung von dieser Startzelle aus erzielt werden kann.

DP-Tabelle

Die endgültige Lösung ist durch $S[0, 0]$ gegeben, was die obere linke Ecke der DP-Tabelle ist.



Randbedingungen:

- $j = m - 1$:
 $a[i, j]$
- $i = 0, j < m - 1$:
 $a[i, j] + \max(\rightarrow, \searrow)$
- $i = n - 1, j < m - 1$:
 $a[i, j] + \max(\rightarrow, \nearrow)$
- sonst:
 $a[i, j] + \max(\nearrow, \rightarrow, \searrow)$

Beispiel: Wie man eine Tabelle füllt

$a =$

$S =$

Beispiel: Wie man eine Tabelle füllt

$a =$

$S =$

8	8	6	5	4	3	1	0
10	8	8	5	5	4	2	1
10	10	6	6	4	4	4	0
10	6	6	5	5	4	2	2
6	6	6	5	4	2	2	1

Python-Implementierung: Maximale Belohnung

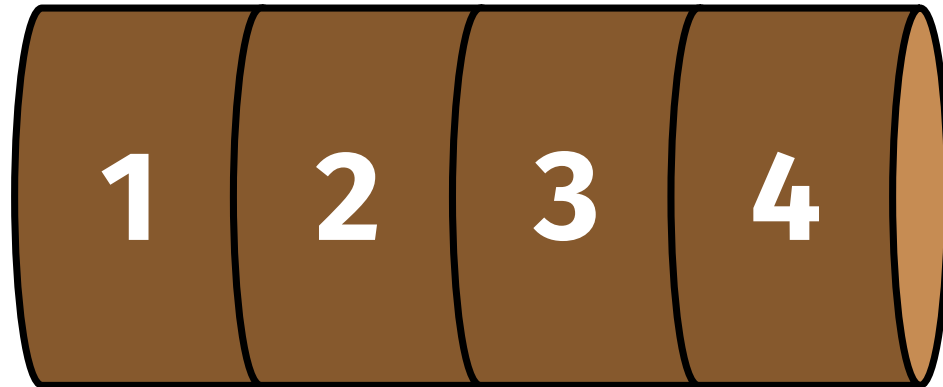
```
import numpy as np

def bestway(a):
    a = np.array(a) # If a is not yet a numpy array
    n,m = a.shape
    S = np.zeros((n,m))

    for j in range(m-1, -1, -1):
        for i in range(n):
            if j == m-1:
                S[i,j] = a[i,j]
            else:
                northeast = S[i-1,j+1] if i > 0 else -1
                east = S[i,j+1]
                southeast = S[i+1,j+1] if i < n - 1 else -1
                S[i,j] = a[i,j] + max(northeast, east, southeast)
```

3. Stab schneiden

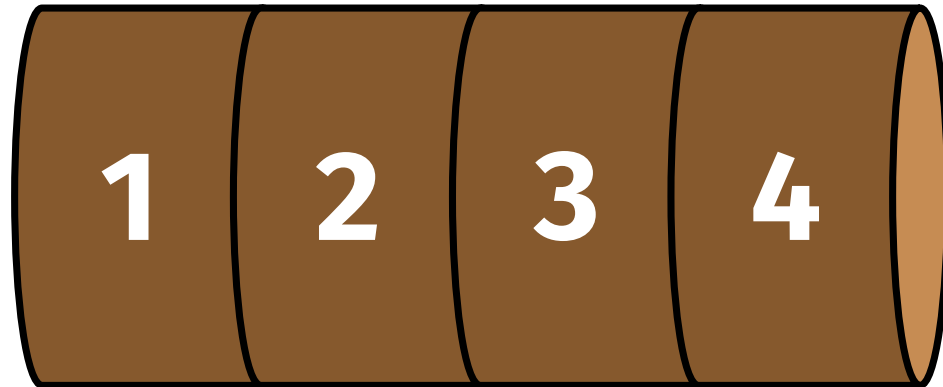
Aufgabenbeschreibung: Stab schneiden



Eingabe:

- Ein Stab der Länge n , den wir zerschneiden. Stellen Sie sich zum Beispiel einen Baum vor.

Aufgabenbeschreibung: Stab schneiden

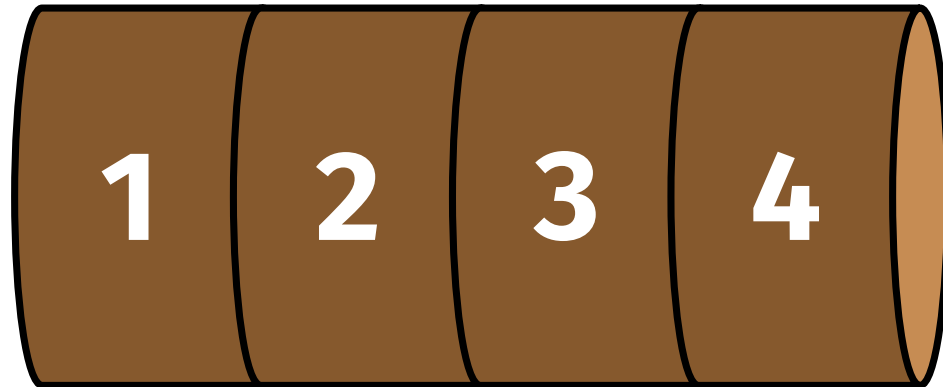


Länge i	0	1	2	3	4
Preis $p[i]$	0	1	5	8	9

Eingabe:

- Ein Stab der Länge n , den wir zerschneiden. Stellen Sie sich zum Beispiel einen Baum vor.
- Eine Preisliste p für Stäbe verschiedener Längen. Wir fügen das offensichtliche Feld $i = 0$ hinzu, damit die Länge dem Index entspricht.

Aufgabenbeschreibung: Stab schneiden



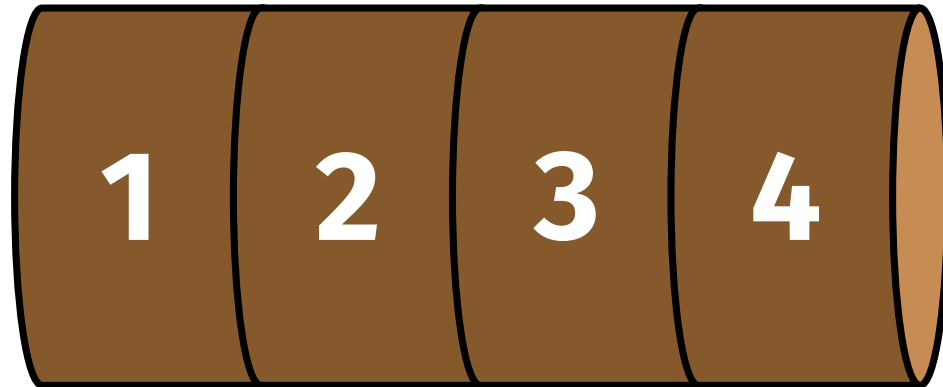
Länge i	0	1	2	3	4
Preis $p[i]$	0	1	5	8	9

Eingabe:

- Ein Stab der Länge n , den wir zerschneiden. Stellen Sie sich zum Beispiel einen Baum vor.
- Eine Preisliste p für Stäbe verschiedener Längen. Wir fügen das offensichtliche Feld $i = 0$ hinzu, damit die Länge dem Index entspricht.

Ziel: Zerschneide den Stab so, dass die Stücke zum höchstmöglichen Preis verkauft werden können.

Aufgabenbeschreibung: Stab schneiden



Länge i	0	1	2	3	4
Preis $p[i]$	0	1	5	8	9

Eingabe:

- Ein Stab der Länge n , den wir zerschneiden. Stellen Sie sich zum Beispiel einen Baum vor.
- Eine Preisliste p für Stäbe verschiedener Längen. Wir fügen das offensichtliche Feld $i = 0$ hinzu, damit die Länge dem Index entspricht.

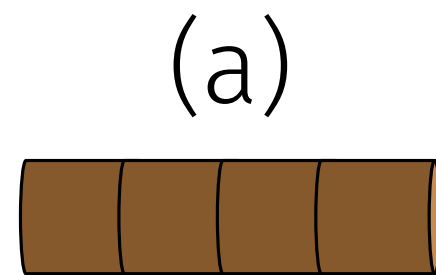
Ziel: Zerschneide den Stab so, dass die Stücke zum höchstmöglichen Preis verkauft werden können.

Ausgabe: Der maximale Wert $S[n]$, der durch das Zerschneiden des Stabes erzielt werden kann.

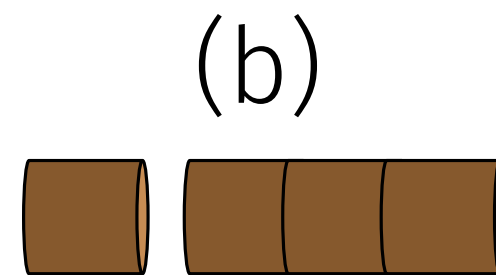
Beispiel

Für einen Stab der Länge 4 gibt es 8 Möglichkeiten, ihn zu zerschneiden:

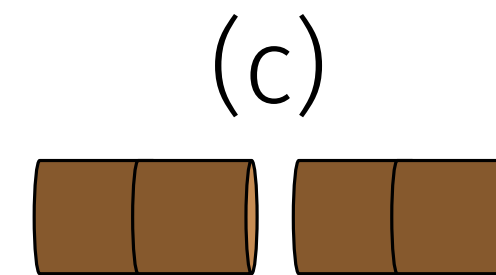
Länge i	0	1	2	3	4
Preis $p[i]$	0	1	5	8	9



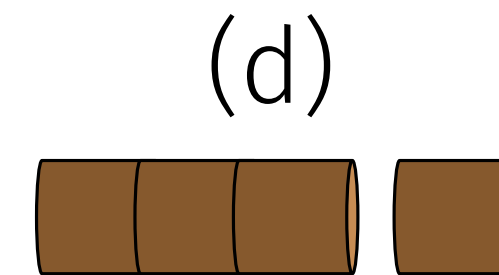
\$9



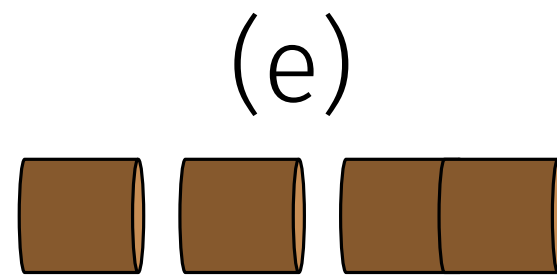
\$1 \$8



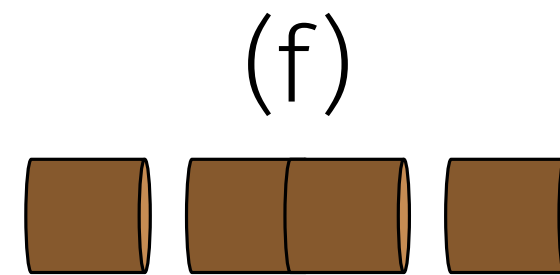
\$5 \$5



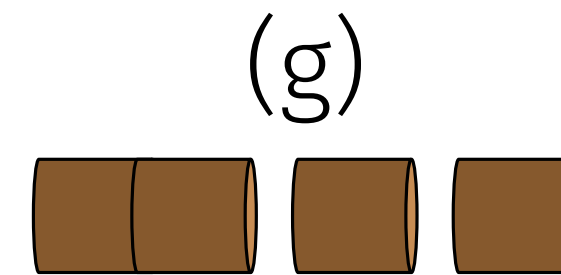
\$8 \$1



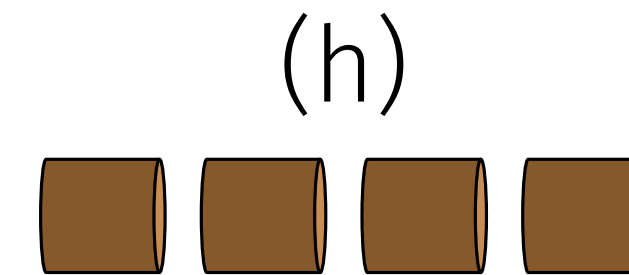
\$1 \$1 \$5



\$1 \$5 \$1



\$5 \$1 \$1



\$1 \$1 \$1 \$1

Wobei (c) mit $\$5 + \$5 = \$10$ den höchsten Gewinn erzielt.

Aufgabenbeschreibung

■ **Problem:** Für einen Stab der Länge n gibt es 2^{n-1} Möglichkeiten.

Bsp: ein Stab der Länge 2, kann man an einer stelle schneiden (oder eben nicht) $\Rightarrow 2^1=2$

Aufgabenbeschreibung

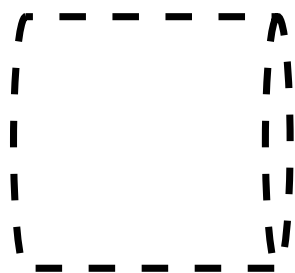
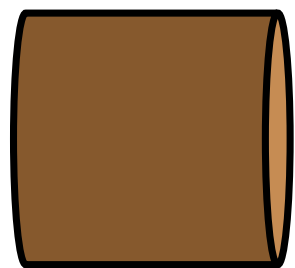
- **Problem:** Für einen Stab der Länge n gibt es 2^{n-1} Möglichkeiten.
→ Um ein Verständnis zu erhalten, beginnen wir mit den kleinsten Längen.

Die drei Schritte der DP

1. **Identifiziere die Teilprobleme** und analysiere, wie sie sich überlappen.
2. **Definiere die rekursive Formel** durch Betrachtung lokaler Übergänge und optimaler Entscheidungen.
3. **Implementiere eine Lösung** mit einer geeigneten Datenstruktur und der korrekten Berechnungsreihenfolge.

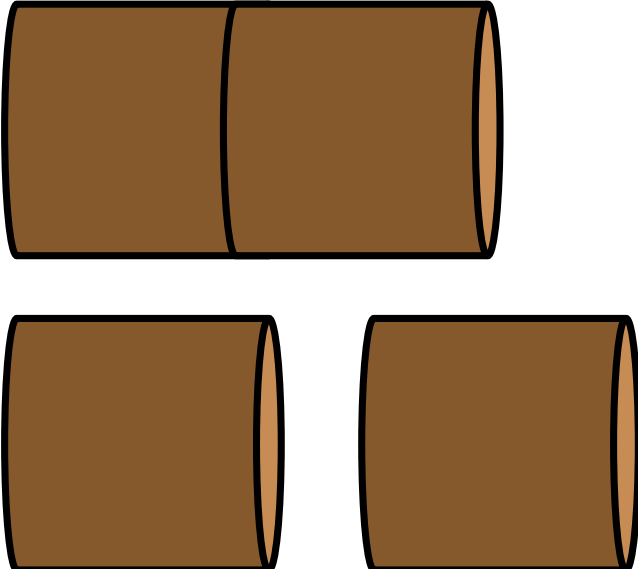
Schritt 1: Identifiziere die Teilprobleme

- Für Stäbe der Länge 0 und 1 haben wir nur jeweils eine Option.

Länge	Optionen	Optimaler Preis
0		0
1		$p[1]$

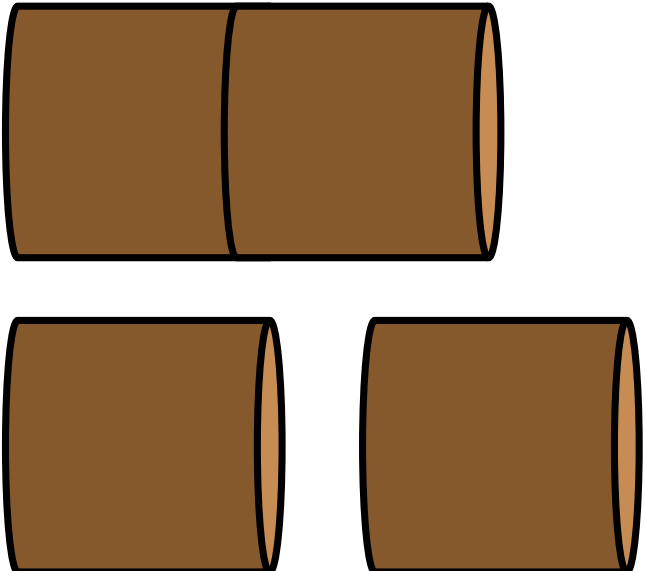
Schritt 1: Identifiziere die Teilprobleme

- Für einen Stab der Länge 2 haben wir bereits mehrere Optionen.

Länge	Optionen	Preisoptionen
0		$p[2]$ $p[1] + p[1]$

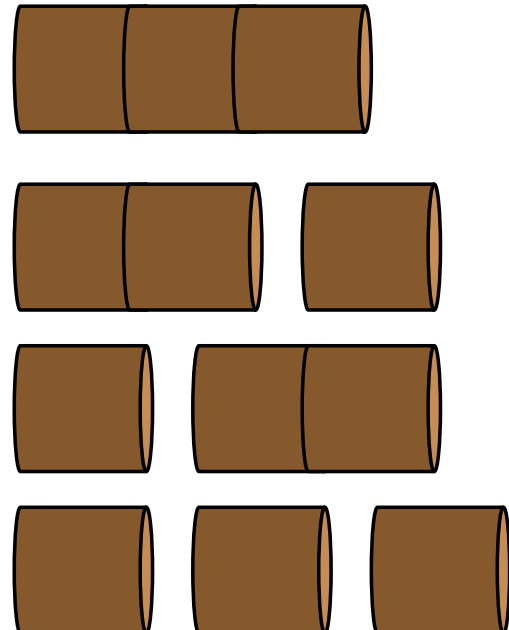
Schritt 1: Identifiziere die Teilprobleme

- Für einen Stab der Länge 2 haben wir bereits mehrere Optionen.
- Die bessere der beiden bestimmt den optimalen Preis.

Länge	Optionen	Optimaler Preis
0		$\max \left\{ \begin{array}{l} p[2] \\ p[1] + p[1] \end{array} \right\}$

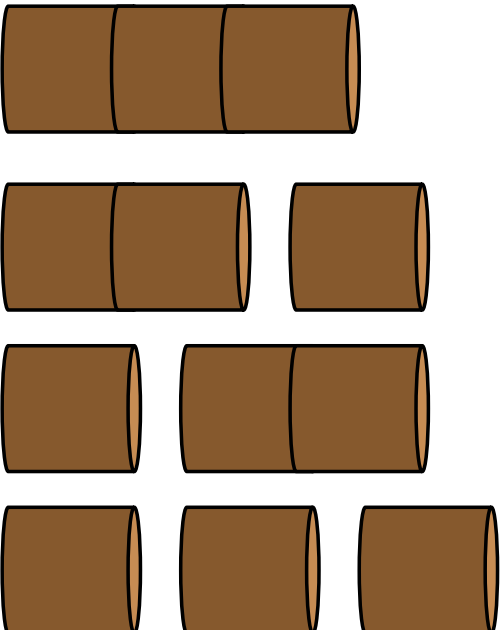
Schritt 1: Identifiziere die Teilprobleme

- Die möglichen Schnitte für einen Stab der Länge 3 hinterlassen einige Stücke mit einer nicht-trivialen Länge von 2.

Länge	Optionen	Preisoptionen
0		$p[3]$ $s[2] + p[1]$ $p[1] + s[2]$ $p[1] + p[1] + p[1]$

Schritt 1: Identifiziere die Teilprobleme

- Die möglichen Schnitte für einen Stab der Länge 3 hinterlassen einige Stücke mit einer nicht-trivialen Länge von 2.
- Der optimale Preis des Stabes der Länge 3 hängt vom optimalen Preis für den Stab der Länge 2 ab.

Länge	Optionen	Optimaler Preis
0		$\max \left\{ \begin{array}{l} p[3] \\ s[2] + p[1] \\ p[1] + s[2] \\ p[1] + p[1] + p[1] \end{array} \right\}$

Schritt 1: Identifiziere die Teilprobleme

- Was sind die Teilprobleme?

Schritt 1: Identifiziere die Teilprobleme

- Was sind die Teilprobleme?

$S[i]$ bezeichnet den optimalen Preis, den man für einen Stab der Länge i erhalten kann. Er hängt von allen $S[j]$ mit $j < i$ ab.

Schritt 2: Definiere die rekursive Formel

- Der optimale Preis eines Stabes der Länge i besteht aus dem Preis des abgeschnittenen Stücks der Länge k und dem Preis des verbleibenden Stücks der Länge $i - k$. Wie sieht das als Formel aus?

Schritt 2: Definiere die rekursive Formel

- Der optimale Preis eines Stabes der Länge i besteht aus dem Preis des abgeschnittenen Stücks der Länge k und dem Preis des verbleibenden Stücks der Länge $i - k$. Wie sieht das als Formel aus?

$$S[i] = p[k] + S[i - k]$$

Preis für den Abschnitt der Länge k

"wie viel bekomme ich mit dem Rest?"

Schritt 2: Definiere die rekursive Formel

- Der optimale Preis eines Stabes der Länge i besteht aus dem Preis des abgeschnittenen Stücks der Länge k und dem Preis des verbleibenden Stücks der Länge $i - k$. Wie sieht das als Formel aus?

$$S[i] = p[k] + S[i - k]$$

- Da die optimale Schnittlänge k unbekannt ist, müssen alle möglichen Optionen bewertet werden, um die beste auszuwählen. Wie machen wir das?

Schritt 2: Definiere die rekursive Formel

- Der optimale Preis eines Stabes der Länge i besteht aus dem Preis des abgeschnittenen Stücks der Länge k und dem Preis des verbleibenden Stücks der Länge $i - k$. Wie sieht das als Formel aus?

$$S[i] = p[k] + S[i - k]$$

- Da die optimale Schnittlänge k unbekannt ist, müssen alle möglichen Optionen bewertet werden, um die beste auszuwählen. Wie machen wir das?

$$S[i] = \begin{cases} 0 & \text{if } i = 0 \\ \max\{p[k] + S[i - k] : k \in [1, i]\} & \text{if } i \neq 0 \end{cases}$$

- Beachte, dass diese Formel nur gilt, wenn p einen Preis für jede Länge von 1 bis i angibt.

Rekursiver Algorithmus

```
def max_value(p, n):
```

```
    #Base case
```

```
    if n == 0:
```

```
        return 0
```

```
    #Optimal Price
```

```
    options = [p[k] + max_value(p, n-k) for k in range(1, n+1)]
```

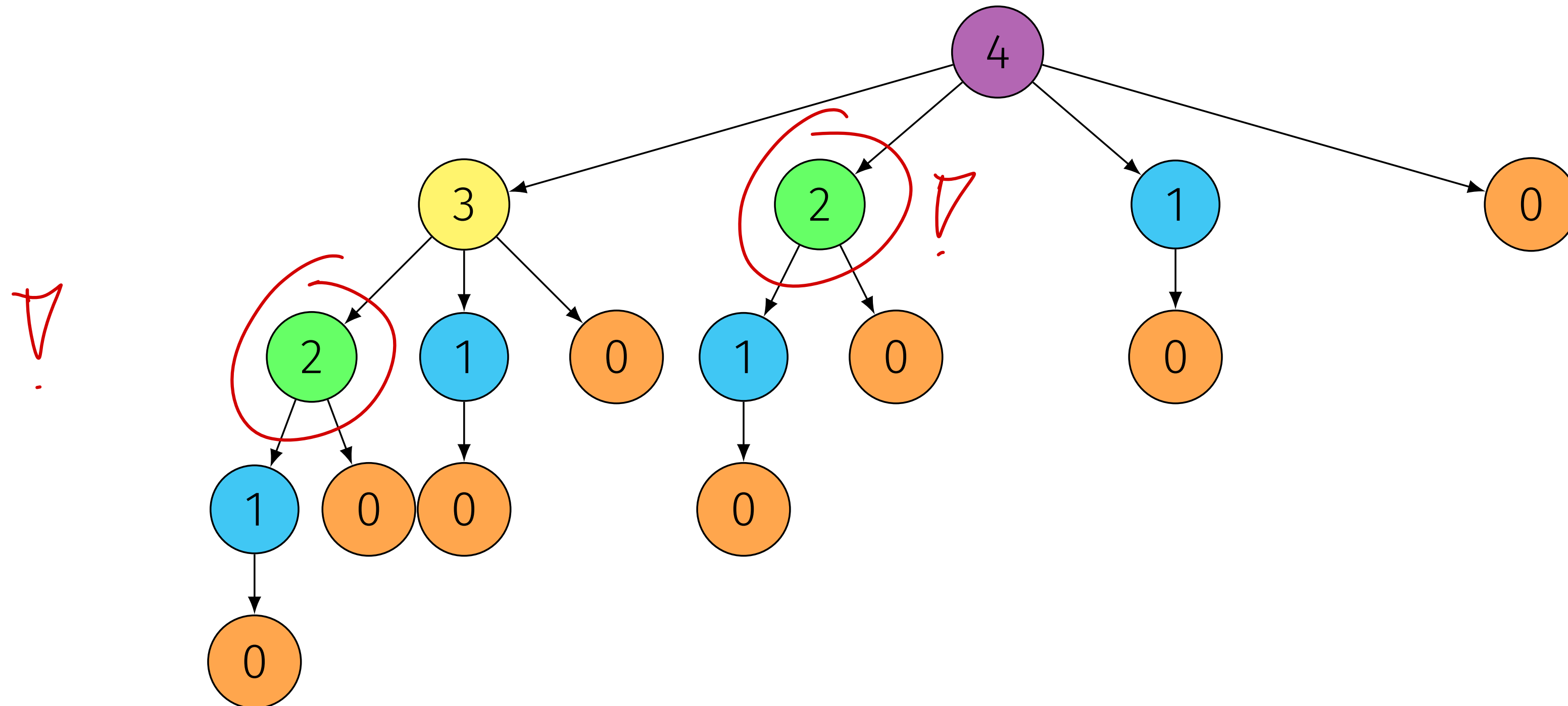
```
    return max(options)
```

obige Formel mit den möglichen k
ausprobieren

|

Überlappende Teilprobleme

Basierend auf dem Beispiel eines Stabes der Länge 4.



- Die Teilprobleme werden mehrfach berechnet! Dies führt zu einer

Dynamische Programmierung

- Ein rekursives Problem mit überlappenden Teilproblemen ist gegeben.
→ Stab schneiden kann daher mittels Dynamischer Programmierung optimiert werden.

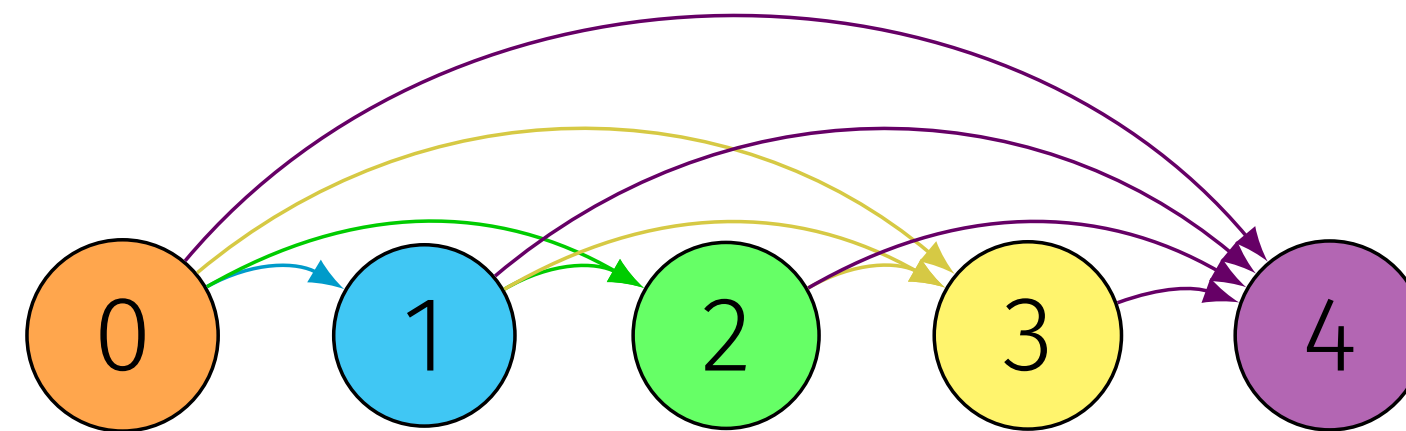
Core-Skill für Inf2: von Rekursion zu (iterativer) DP

Schritt 3: Implementierung

- Bestimme eine geeignete Datenstruktur:
 - 1D-Array der Länge $n + 1$ — für die 0 „irgendwie speichern was wir für eine gewisse länge max bekommen“
- Identifiziere die Abhängigkeiten:
 - Um $S[i]$ zu berechnen, müssen wir zuerst $S[i - 1], \dots, S[i - k]$ kennen.
 - Letztendlich hängt alles von $S[0]$ ab, dem Basisfall.
- Identifiziere die korrekte Berechnungsreihenfolge:
 - Aus den Abhängigkeiten wissen wir, dass wir bei $i = 0$ beginnen und das Array von links nach rechts ausfüllen müssen: Von kleineren zu grösseren Schnitten.

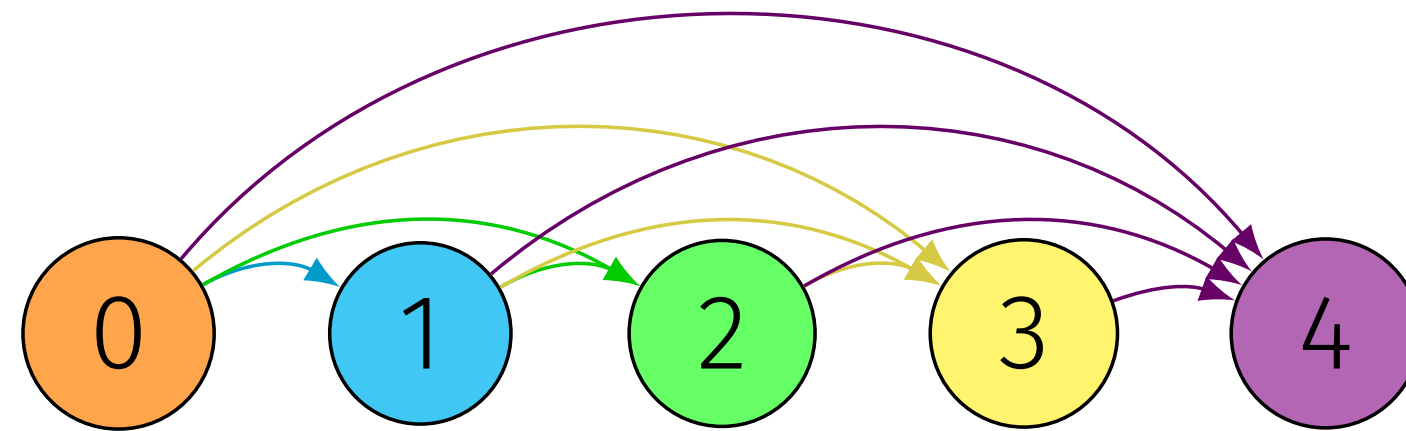
Dynamische Programmierung: Theorie

- $S[3]$, der optimale Preis eines Stabes der Länge 3, hängt von $S[2]$ und $S[1]$ ab, sowie vom Basisfall $S[0]$.
- $S[2]$ hängt von $S[1]$ und $S[0]$ ab.
- $S[1]$ hängt nur von $S[0]$ ab.



Dynamische Programmierung: Theorie

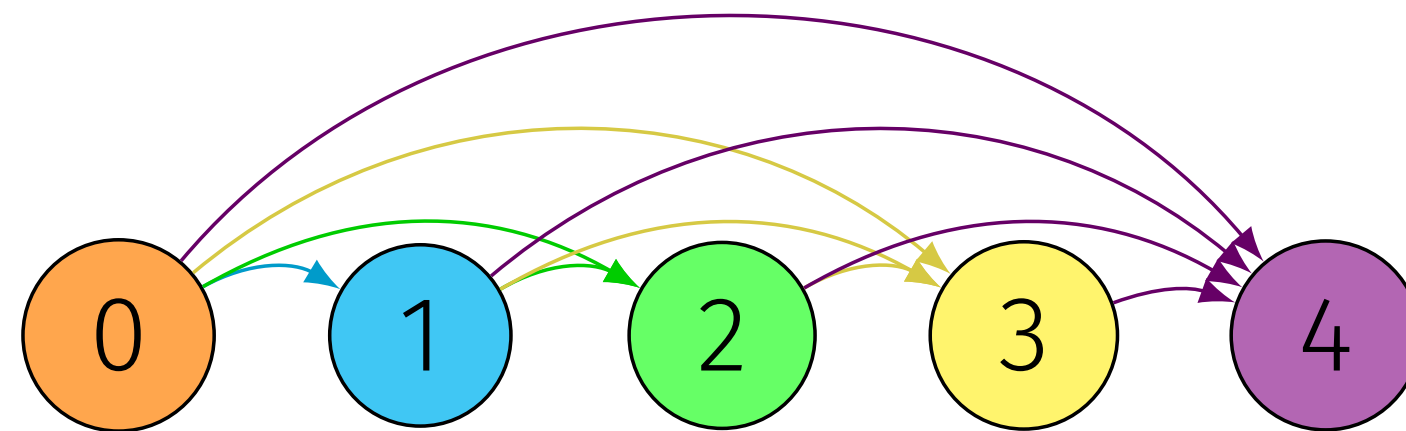
- $S[3]$, der optimale Preis eines Stabes der Länge 3, hängt von $S[2]$ und $S[1]$ ab, sowie vom Basisfall $S[0]$.
- $S[2]$ hängt von $S[1]$ und $S[0]$ ab.
- $S[1]$ hängt nur von $S[0]$ ab.



- Da $S[0] = 0$ bereits bekannt ist, beginnen wir mit $S[1]$ und berechnen die Ergebnisse Schritt für Schritt bis zu $S[n]$.

Dynamische Programmierung: Theorie

- $S[3]$, der optimale Preis eines Stabes der Länge 3, hängt von $S[2]$ und $S[1]$ ab, sowie vom Basisfall $S[0]$.
- $S[2]$ hängt von $S[1]$ und $S[0]$ ab.
- $S[1]$ hängt nur von $S[0]$ ab.



- Da $S[0] = 0$ bereits bekannt ist, beginnen wir mit $S[1]$ und berechnen die Ergebnisse Schritt für Schritt bis zu $S[n]$.
- Es gibt keine Schleifen im Graphen. Wir nennen ihn gerichtet azyklisch. Sie werden dieses Muster in allen DP-Problemen bemerken, die wir präsentieren

Dynamische Programmierung

Mit diesem Wissen können wir leicht eine "Richtung" finden, um die Probleme zu lösen!

```
import numpy as np
def max_value_dp(p, n):
```

```
    # Create data structure
```

```
    S = np.zeros(n+1)
```

oder $[0] \cdot (n+1)$

```
    # Implement Base case (redundant through np.zeros)
```

```
    S[0] = 0
```

```
    # Fill the data structure in the appropriate direction.
```

```
    for i in range(1, n+1):
```

```
        options = [p[k] + S[i-k] for k in range(1, i+1)]
```

```
        S[i] = max(options)
```

```
    return S[n]
```

bekannte Formel

Memoisierung

→ In rot die Änderungen am naiven rekursiven Algorithmus.

```
def max_value(p, n, memo = None):  
    # Create data structure if not provided  
    if memo is None:  
        memo = dict()  
  
    # Compute subproblem only if it hasn't been solved  
    if n not in memo:  
  
        if n == 0:  
            memo[0] = 0  
        else:  
            options = [p[k] + max_value(p, n-k, memo)  
                       for k in range(1, n+1)]  
            memo[n] = max(options) # Store the result  
  
    return memo[n]
```

4. Zusammenfassung

Zusammenfassung: Dynamische Programmierung

Die drei Schritte der DP

1. **Identifiziere die Teilprobleme** und analysiere, wie sie sich überlappen.
2. **Definiere die rekursive Formel** durch Betrachtung möglicher Übergänge zwischen Teilproblemen und optimaler Entscheidungen.
3. **Implementiere eine Lösung** mit einer geeigneten Datenstruktur und der korrekten Berechnungsreihenfolge.

Zusammenfassung: Dynamische Programmierung

Allgemeiner Algorithmus für dynamische Programmierung:

1. Initialisiere die Datenstruktur
2. Implementiere den Basisfall
3. Fülle die Datenstruktur aus
4. Gib das Ergebnis zurück

Immer machen, bringt euch oft auf die richtige Spur zur Lösung!

Zusammenfassung: Königsweg

1. Initialisiere Datenstruktur:

```
S = np.zeros((n,m))
```

- 2D-Matrix

2. Implementiere Basisfall:

```
S[:, m] = a[:, m]
```

- Äusserste rechte Spalte

3. Fülle die Datenstruktur aus:

- Von rechts nach links

```
for j in [...]:  
    for i in [...]:  
        opt1 = [...]  
        opt2 = [...]  
        opt3 = [...]  
        S[i, j] = max(opt1, opt2, opt3)
```

4. Gib Ergebnis zurück

```
return S[0,0]
```

Zusammenfassung: Stab schneiden

1. Initialisiere Datenstruktur:

```
S = np.zeros(n+1)
```

- 1D-Array

2. Implementiere Basisfall:

```
s[0] = 0
```

- Stab der Länge 0

3. Fülle die Datenstruktur aus:

```
for i in [...]:  
    options = [... for k in cuts]  
    S[i] = max(options)
```

- Variable Anzahl an Optionen

4. Gib Ergebnis zurück

```
return S[n]
```


Mech 2 Midterm

- Aufgaben in den Serien deutlich komplizierter als die an der Prüfung
- Achtet auf alle Kniffe & Tricks in den ML, die Zeit ist knapp!
 - „Exposure“ zu alten Prüfungen entsprechend sehr wertvoll!
- Vermutlich an der BP besser als an der ZP, aber es ist eine gute Absicherung vor einem Ausrutscher im Sommer.
- **Beanspruchen beherrschen!**

Übung 9: DP I

Übung 9: DP I

- Tribonacci

- Catalan-Zahlen

Alle wärmstens Empfohlen!

- Blöcke sind Aufgaben wo die Zeit ins Verständnis gut investiert ist! Versucht, euch dafür zu begeistern!

- Aufgabenplanung 2.0

Abgabedatum: Montag 04.05.2026, 20:00 MEZ

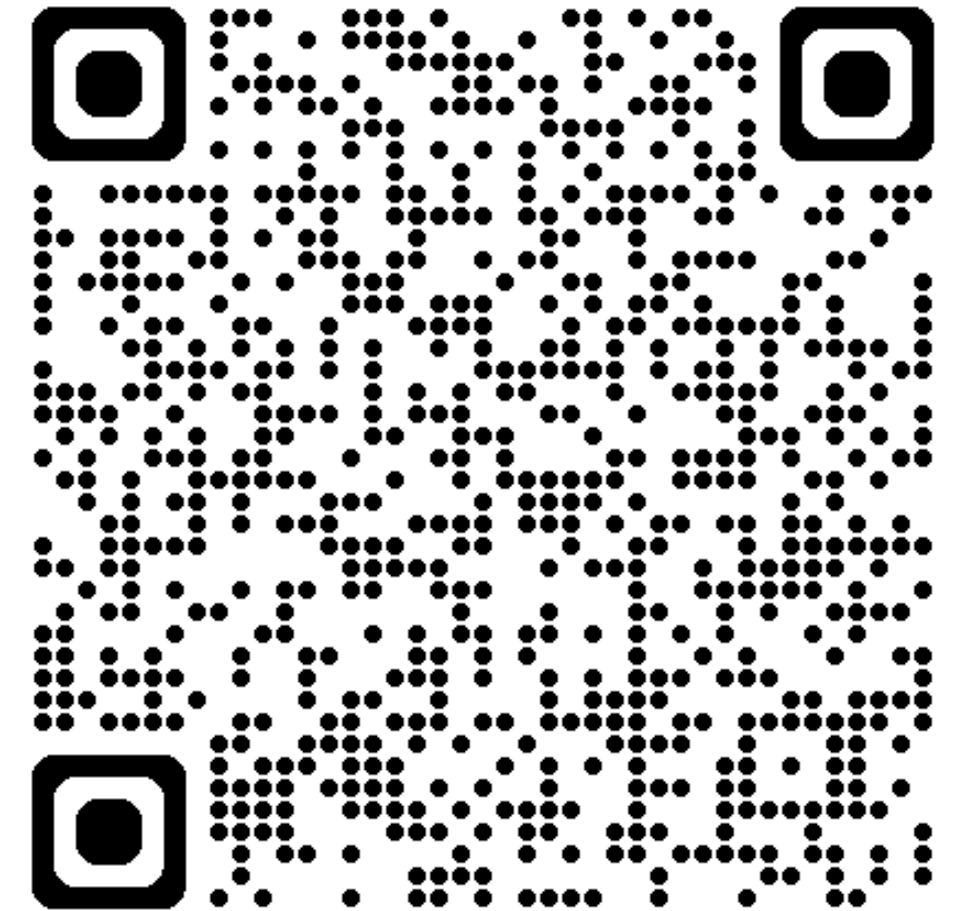
KEINE HARDCODIERUNG

Bis nächste Woche

...nächste Woche:

- **Dynamic Programming**

Anonymes Feedback Formular



Gerne Feedback: Was soll ich beibehalten/ändern

& Themenwünsche, die ich in den letzten 2 Wochen nochmals anschauen soll. (ihr könnt auch auf CX einen kommentar schreiben)

& Feedback zu Slides in Goodnotes